

 01-Introduzione alle Tecnologie Web	1
 Concetti principali	1
 Il browser	1
 Programmazione lato client	1
 Il server web	1
 Organizzazione	1
 Dove si trova il server web?	1
 Architetture complesse	1
 Chi usa il web?	1
 02-Programmazione Lato Server e CGI	1
 Cos'è la programmazione lato server	1
 Utilità	2
 Limiti (senza programmazione client-side)	2
 Input dell'utente	2
• Quando usare GET o POST?	2
 I cookie	2
 Caratteristiche dei cookie:	2
 Tecniche lato server	2
 CGI – Common Gateway Interface	2
 Interazione CGI	2
 Posizione dei CGI	2
 Input per i CGI	2
 Output di un programma CGI	2
 Linguaggi per CGI	2
 Esempi pratici	3
 Conclusione	3
 03-Cloud Computing	3
 Cos'è il Cloud Computing	3
 Esempi di Cloud	3
 Tipologie di Nuvola	3
 Perché usare il cloud	3
 Soluzione: Cloud	3
 Casi d'uso	3
 Tecnologie correlate	3
 Modelli di servizio nel Cloud	4
 Differenze:	4
 Virtualizzazione e Astrazione	4

 Attori del Cloud	4
• Vantaggi	4
 Problemi e rischi	4
 Indicazioni del Garante Privacy	4
IT Strategia Cloud Italia (PA)	4
 Piano Triennale per l'Informatica nella PA	4
 Censimento Data Center PA (2018-19)	4
 Migrazione al cloud della PA	4
 Classificazione dei dati	4
 Polo Strategico Nazionale (PSN)	5
 04-Amaزون e il Cloud	5
 Principi di AWS	5
 Costi	5
 Architettura AWS	5
 Sicurezza	5
 Servizi AWS principali	5
Computazione	5
Memorizzazione	5
Database	5
Reti	5
Content Delivery	5
Servizi applicativi	5
 Dettaglio servizi	6
Amazon S3	6
Amazon CloudFront	6
Amazon EC2	6
Database gestiti	6
Elastic MapReduce	6
 Strumenti di gestione e sviluppo	6
Gestione	6
Sviluppo	6
 Esempi di utilizzo	6
• Conclusione	6
 05-Amaزون EC2: cos'è	6
 Scelta AMI	7
 Creazione e accesso alle istanze	7
 Uso di sudo	7
 Tipologie di istanze	7
 Persistenza dei dati	7
 Costi EC2	7
 Costi EBS e banda	7

 CloudFormation	7
 Istanze Spot	7
 Funzioni aggiuntive	8
Elastic Load Balancer (ELB)	8
CloudWatch	8
Auto Scaling	8
 Conclusione operativa	8
 06-EC2 Web Server	8
 Attivazione e accesso all'istanza EC2	8
 Apache: concetti base	8
 Struttura file e directory in Apache	8
 Configurazione tipica Apache	8
Gestione processi:	9
 Privilegi dei file	9
Uso di chmod:	9
 Installazione di Apache e PHP su Ubuntu	9
 Alternative locali per test	9
• Sintesi operativa:	9
 07-Scripting lato server e PHP	9
 Esecuzione degli script PHP	9
 PHP: Introduzione	10
 Sintassi e basi PHP	10
 Gestione form e parametri utente	10
 Funzioni PHP	10
 Intestazioni HTTP	10
 Cookie	11
08-PHP-Gallery	11
Obiettivi di apprendimento	11
Struttura dell'applicazione	12
File config.php	12
File funzioni.php	12
index.php (pagina indice)	12
visualizza.php (visualizzazione immagine)	12
inserisci.html (form upload)	12
inserisci.php (gestione upload)	12
thumbnail.php (miniature on-the-fly, opzionale)	12
Funzioni PHP e Filesystem utilizzate	13
Sicurezza	13
GD Library	13

09-PHP-eshop-sessioni	13
Versione 1 (senza carrello).....	13
Semplificazioni	13
Struttura dell'applicazione.....	13
Memorizzazione prodotti (shop.txt).....	13
Funzioni principali (funzioni.php).....	14
Sessioni e autenticazione	14
Struttura autenticazione	14
Versione 2 (aggiunta del carrello con sessioni).....	14
Funzionalità aggiunte:	14
Cosa aggiungere:.....	14
Note tecniche importanti.....	14
• Funzioni PHP utilizzate:	14
• Sessioni:	14
• Sicurezza:	14
• Con questo modulo ora sai:.....	15
10-PHP-OO-Composer	15
PHP Object-Oriented Programming (OOP)	15
Perché usarla:	15
Sintassi:	15
Namespaces	15
Gestione di librerie esterne in PHP	15
Composer	16
Uso pratico di Composer	16
Passaggi operativi:	16
Distribuzione del progetto con Composer	16
Sintesi finale:	16
11-SOA Cos'è un Servizio	17
Service Orientation (SO)	17
Definizione:.....	17
Principi della SO	17
Service Oriented Computing (SOC)	17
Obiettivi e benefici SOC	17
Service Oriented Architecture (SOA)	17
Cos'è:	17
Implementazione SOA:.....	17
Ruoli nella SOA.....	17
Operazioni nella SOA.....	17
Publish:.....	17
Find:	17
Bind:	18

Esempio pratico SOA	18
Livelli di adozione SOA	18
Quality of Service (QoS)	18
Definizione:.....	18
Dimensioni:.....	18
SLA (Service Level Agreement)	18
Elementi SLA:	18
SOA nella pratica	18
Sintesi operativa:	18
12-WS1Cos'è un Web Service	19
Definizione (W3C):	19
In sintesi:	19
Attori principali nei WS	19
Applicazioni tipiche	19
Tecnologie abilitanti dei WS	19
Standard di base:	19
Altri standard:.....	19
Comunicazione nei WS	19
SOAP (Simple Object Access Protocol).....	19
WSDL (Web Services Description Language)	20
WS-Policy	20
Web Services in PHP	20
Esempio pratico in PHP:.....	20
Sintesi:	20
13-REST-JSON, REST: cos'è	21
RESTful Web Services	21
JSON (JavaScript Object Notation)	21
Procedura REST	21
Esempi in PHP	21
JSON:.....	21
REST client minimale:.....	21
REST vs SOAP	21
REST:.....	21
SOAP:.....	21
Caso di studio REST: Sistema informativo aziendale	21
Problematiche REST in PHP	21
URL rewriting per REST.....	22
service.php REST handler in PHP	22
Risposte in XML o JSON	22

XML:	22
JSON:	22
Client REST	22
Documentazione dei servizi REST	22
Sintesi finale:.....	22
15-Cloud AWS S3: cos'è	22
S3 Glacier	22
SDK AWS per PHP	23
Problemi comuni SDK:	23
Gestione delle credenziali AWS.....	23
Prerequisiti pratici.....	23
Esercizi in PHP con AWS S3.....	23
Esercizio 1: Elenco dei bucket	23
Esercizio 2: Elenco oggetti nei bucket.....	23
Esercizio 3: Upload file su S3	23
Esercizio 4: Integrazione con CloudFront.....	23
Attenzioni pratiche:	23
Upload sicuro con Exponential Backoff:	24
Altre funzioni S3	24
Sintesi:	24
16-AWS-DynamoDB, Introduzione a dati e database	24
Modelli di dati	24
AWS e database	24
DynamoDB: esercitazioni pratiche in PHP	24
Setup iniziale:.....	24
Inserimento dati:	25
Query dati:	25
Scan con filtro:	25
Creazione tabella via PHP:	25
Note pratiche per debug.....	25
Monitoraggio costi e attività.....	25
Sintesi finale:.....	25
17-Cloud-altro, Azure (Microsoft)	26
Google Cloud.....	26
Heroku	26
Cos'è:.....	26
Tool forniti:	26
Ciclo di sviluppo:	26
Cedar:.....	26

 Git e Heroku:	26	 Sintesi finale:.....	30
 Ambienti multipli:.....	26	 20-Cloud-IoT-BigData – Internet of Things	31
Slug:	26	Tecnologie essenziali per IoT:	31
Dyno:	26	 Architettura Cloud + IoT	31
Integrazione con S3:	26	 Caso di studio: The Things Network (TTN)	31
 Add-ons:	27	Altre piattaforme e reti:	31
 Pro e Contro Heroku:	27	 Dashboard e Analytics IoT	31
• Pro:	27	 Esempio locale: PollicIoT	31
• Contro:	27	 Big Data	31
 Sintesi finale:	27	 Computazione in cloud	32
 18-Cloud-Lambda, Serverless Computing	27	GPU in cloud:	32
 Paradigma cloud	27	GPU gratis o quasi:	32
 Deployment semplificato	27	 Storage per Big Data	32
 Event-driven	27	 Elaborazione e analisi dati in cloud	32
 Function as a Service (FaaS)	27	 GPU: cloud o hardware?	32
 AWS Lambda.....	27	 Sintesi finale.....	32
 Costi AWS Lambda	27		
 Casi d'uso	28		
 Orchestrazione con Step Functions	28		
 Composizione di Web Services	28		
Modalità:.....	28		
 Esercizio pratico Lambda + S3	28		
Obiettivo:	28		
Passaggi:.....	28		
Codice Lambda:.....	28		
 Sintesi finale.....	29		
 19-AV-Assistenti Vocali: introduzione	29		
 Sviluppare per assistenti vocali	29		
Struttura delle app vocali:.....	29		
 Alexa Skills vs Google Actions	29		
 Creazione di una Alexa Skill	29		
 Struttura di una skill.....	30		
Handler.....	30		
 Esempio di handler.....	30		
 Slots	30		
 Esempio completo di skill per il corso TWC	30		
 Handlers utili.....	30		
 Costruzione della skill	30		
 Visualizzazione su schermo	30		

01-Introduzione alle Tecnologie Web

Concetti principali

1. Client/Server e HTTP

- Il **web** si basa sull'interazione tra un **client** (browser) e un **server**.
- La comunicazione avviene tramite il **protocollo HTTP**, strutturato in:
 - **Header** (intestazione): informazioni su metodo (GET, POST...), URI, versione protocollo, metadati.
 - **Body** (corpo): può contenere dati utente, tipicamente nei POST.
- Esempi di intestazioni includono *User-Agent*, *Accept*, *Referer*, ecc.

2. Risposta HTTP

- Composta anch'essa da header (con codici di stato: 200 OK, 404 NOT FOUND, ecc.) e body (contenente la risorsa richiesta, es. una pagina HTML).
- Include metadati come *Content-Type* e *Content-Length*.

Il browser

- È il client che:
 - Interpreta le richieste dell'utente.
 - Invia richieste HTTP al server.
 - Visualizza o esegue la risposta.
- Può gestire risorse tramite:
 - **Codice interno**: HTML, immagini, ecc.
 - **Plugin**: per gestire formati specifici.
 - **Helper esterni**: programmi esterni invocati automaticamente.

Programmazione lato client

- Esecuzione di programmi nel browser → **sicurezza e limitazioni**
- Approcci:
 - **Scripting** (JavaScript/ECMAScript + DOM)
 - **Applet Java** (meno usate oggi)
 - **Flash** (obsoleto)

Il server web

- È un **daemon** sempre attivo.
- Risponde alle richieste distribuendo:
 - **Risorse statiche** (HTML, immagini).
 - **Risorse dinamiche**, generate al momento (es. tramite form).
- Funzionalità aggiuntive:
 - Controllo permessi e autenticazione.
 - Logging degli accessi/eventi.

Organizzazione

- Le risorse si trovano in un **sottoalbero** del filesystem, definito nella configurazione.
- Le richieste URL vengono mappate a file o script in questo albero.

Dove si trova il server web?

- Deve garantire disponibilità continua:
 - **On-premise**: in azienda.
 - **Hosting**: ospitato da un provider.
 - **Housing**: server aziendale fisicamente nei locali del provider.
 - **Cloud**: infrastruttura virtuale.

Architetture complesse

- Nelle applicazioni più grandi:
 - Si usano **più server**: es. web server + database.
 - Si scalano **orizzontalmente**.
 - Si adottano **architetture distribuite e service-oriented**.

Chi usa il web?

- **Utenti umani**: tramite browser.
- **Altri programmi**: tramite **Web Services**, che seguono il paradigma **Service-Oriented Computing**.

02-Programmazione Lato Server e CGI

Cos'è la programmazione lato server

- Tecnica per **creare risorse dinamiche** da distribuire tramite un server web.
- I **programmi generano contenuti** (HTML, CSS, immagini...) al momento della richiesta.
- Sono **attivati dal web server** su richiesta di un client.

Utilità

- Interfacce web a **basi di dati** (es. login, registri).
- **Contenuti dinamici** in base al contesto (utente, data, browser...).
- **Applicazioni interattive** (simili a quelle desktop, ma limitate).
- Le pagine web sono spesso **costruite da dati strutturati** (es. database).

Limiti (senza programmazione client-side)

- Scarsa interattività:
 - Ogni input utente → invio al server → nuova risposta HTML → sostituisce l'interfaccia.
- Mancanza di sessione (senza cookie): ogni pagina è “scollegata” dalle altre.

Input dell'utente

- Fornito tramite **form HTML**.
 - Parametri trasmessi al server:
 - **GET**: visibili nell'URL (`?nome=Mario&cognome=Rossi`)
 - **POST**: nel **corpo** della richiesta HTTP.
 - Il server riceve i dati e li **passa al programma**, che genera la risposta.
- ### • Quando usare GET o POST?
- **GET**: utile per bookmarking, ma poco sicuro (password visibili, limiti di lunghezza).

- **POST**: preferibile per dati riservati o grandi (es. upload file).

I cookie

- Inventati da Netscape, servono per **memorizzare dati sul client**.
- Utili per:
 - **Persistenza tra visite** (es. sessioni).
 - **Tracciamento utenti**.
- Il server può inviare cookie tramite l'intestazione Set-Cookie.

Caratteristiche dei cookie:

- **Dominio e path** di validità.
- **Data di scadenza**.
- **Nome e valore** (stringa).
- Memorizzati dal browser in file o memoria.

Tecniche lato server

- Differiscono per:
 - Come interagiscono con il server.
 - Dove risiede il codice.
 - Modalità di esecuzione.

CGI – Common Gateway Interface

- Prima tecnica storica per pagine dinamiche.
- Il server **esegue un programma** (script) al momento della richiesta.
- Gestita in Apache da `mod_cgi`.

Interazione CGI

- Riceve **parametri utente**.

- Costruisce l'intera **risposta HTTP**.
- Ogni richiesta → **nuovo processo**, quindi è **computazionalmente costosa**.

Posizione dei CGI

- Salvati in directory **dedicata e protetta** (es. `/var/www/cgi-bin/`).
- Non devono mai essere scaricabili dal client (sicurezza!).

Input per i CGI

- Accesso tramite:
 - **Variabili d'ambiente** (es. `QUERY_STRING`, `PATH_INFO`, `CONTENT_TYPE`, `REMOTE_ADDR`).
 - **Standard input** (nel caso di POST).

Output di un programma CGI

- Deve **produrre una risposta HTTP valida**, almeno con:
- `Content-type: text/html`
- Il contenuto può essere HTML, immagini, ecc.

Linguaggi per CGI

- Qualsiasi linguaggio che accede a variabili d'ambiente può essere usato.
- **Perl** è il più comune per CGI (ottima gestione di stringhe).
- Vanno bene anche **C**, **Shell Unix**, ecc.

Esempi pratici

1. Hello World CGI in Perl

```
#!/usr/bin/perl
print "Content-type: text/html\r\n\r\n";
print "<html><body><p>Ciao,
Plutone.</p></body></html>";
```

2. CGI con form HTML (registro visitatori)

Form HTML:

```
<form method="post" action="cgi-bin/registro.pl">
  Nome: <input type="text" name="nome" />
  Cognome: <input type="text" name="cognome" />
  Email: <input type="text" name="email" />
  <input type="submit" value="Invia" />
</form>
```

Script Perl (semplificato):

- Legge i parametri (POST o GET)
- Divide in coppie chiave=valore
- Usa uri_unescape() per decodificare
- Salva su file visitatori.txt
- Mostra una **pagina HTML con i dati**

Conclusione

Il modulo illustra i **fondamenti della programmazione lato server**, in particolare tramite **CGI**, chiarendo:

- Struttura delle richieste HTTP.
- Ruolo del server.
- Modalità di gestione input/output.
- Esempi concreti in **Perl** per costruire applicazioni dinamiche.

03-Cloud Computing

Cos'è il Cloud Computing

- Tecnologie e processi per **virtualizzare e astrarre risorse informatiche**:
 - Memoria, potenza di calcolo, rete, applicazioni.
- ICT vista come una **utility** (come luce, gas).
- Uso flessibile e **pay-per-use**: pago ciò che uso, quando mi serve.

Esempi di Cloud

- **Consumer**: Dropbox, Google Drive, iCloud, OneDrive.
- **Applicazioni Web**: Google Docs, Office365.
- **Cloud provider**: Amazon AWS (S3, EC2), Microsoft Azure, Force.com.

Tipologie di Nuvola

- **Cloud pubblico**: accessibile a tutti, pagamento a consumo.
- **Cloud privato**: uso interno ad un'organizzazione.
- **Cloud ibrido**: combinazione dei due.

Perché usare il cloud

- Le risorse locali sono limitate e costose da aggiornare.
- I server tradizionali sono spesso **sottoutilizzati** (5-20%).
- Problemi di **dimensionamento**: se dimensiono per il picco → spreco.

Soluzione: Cloud

- Risorse **noleggiate al bisogno**, elasticità massima.
- "1000 server per 1 ora = 1 server per 1000 ore"
- Computazione e dati si **spostano su Internet**.

Casi d'uso

- Carichi variabili e imprevedibili (startup, e-commerce).
- Alte esigenze computazionali in poco tempo.

Tecnologie correlate

- **Cluster**: nodi centralizzati e gestiti unificatamente.
- **Grid**: risorse distribuite tra organizzazioni autonome.
- **CDN** (Content Delivery Network): distribuzione efficiente di contenuti statici vicini all'utente.
- **P2P, ASP, Utility computing**: concetti affini.

Modelli di servizio nel Cloud

Modello	Descrizione	Esempi
SaaS (Software as a Service)	Software utilizzato via web	Google Docs, Salesforce
PaaS (Platform as a Service)	Piattaforma software remota per sviluppare	Google App Engine, Heroku
IaaS (Infrastructure as a Service)	Risorse di base (server, storage)	Amazon EC2, DigitalOcean

Differenze:

- **PaaS** = meno controllo ma gestione semplificata.
- **IaaS** = massimo controllo, ma maggiore responsabilità.

Virtualizzazione e Astrazione

- Maggiore astrazione = più facilità di scalare.
- Esempi:
 - **Amazon EC2** = risorsa simile all'hardware → meno scalabile.
 - **Google AppEngine** = modello rigido ma altamente scalabile.

Attori del Cloud

- **Cloud provider:** fornisce SaaS, PaaS, IaaS.
- **Configuratore/Amministratore:** compone e configura i servizi.
- **Utente finale:** utilizza le applicazioni.

• Vantaggi

- Aggiornamenti centralizzati, no obsolescenza.
- Scalabilità, efficienza, zero manutenzione per l'utente.
- Aumento offerta e qualità dei servizi digitali (per PA, PMI, cittadini).

Problemi e rischi

- **Vendor lock-in:** difficoltà di migrazione.
- **Privacy:** dove sono i dati? Quale legge li tutela?
- **Sicurezza:** credenziali in vendita sul dark web, accesso da provider multipli.
- **Dipendenza dalla connettività Internet.**

Indicazioni del Garante Privacy

- Valutare attentamente rischi e benefici.
- Verificare affidabilità e sede del provider.
- Leggere le clausole contrattuali.
- Adottare misure di sicurezza.
- Formare il personale.

IT Strategia Cloud Italia (PA)

Piano Triennale per l'Informatica nella PA

- Obiettivi:
 - **Cloud first:** adozione prioritaria del cloud.
 - Servizi **inclusivi, interoperabili, open source.**
 - Protezione **by design.**
 - Miglioramento **agile e continuo.**
 - Condivisione dei dati pubblici.

- **Once only:** evitare ridondanze informative.

Censimento Data Center PA (2018-19)

- Classificazione:
 - **Gruppo A:** data center avanzati (27).
 - **Gruppo B:** da dismettere e migrare (1190).
- Requisiti per Gruppo A: ISO, affidabilità 99.98%, antincendio, accesso controllato...

Migrazione al cloud della PA

- Due strade:
 1. **Polo Strategico Nazionale (PSN):** cloud privato per dati critici.
 2. **Cloud pubblico certificato:** per dati ordinari.

Strategia di migrazione

- Tipologie: Retire, Retain, Rehost, Repurchase, Replatform, Re-architect.
- Supporto alle PA tramite strumenti, programmi, linee guida.

Classificazione dei dati

- **Strategici:** sicurezza nazionale (es. difesa, bilancio).
- **Critici:** salute, sicurezza, benessere economico.
- **Ordinari:** es. portali informativi.



Polo Strategico Nazionale (PSN)

- Creato dal Dipartimento per la trasformazione digitale.
- Partner: TIM, CDP, Leonardo, SOGEL.
- Ospita dati e servizi critici per:
 - 200+ PA centrali, ASL, Regioni, grandi Comuni.



Benefici del cloud per:

- **Pubblica Amministrazione:**
 - Risparmi, efficienza, migliore offerta servizi, sostenibilità.
- **Cittadini:**
 - Servizi migliori, affidabili, semplici e sicuri.
- **PMI:**
 - Opportunità per offrire servizi cloud alla PA.
 - Incentivo a investire nel settore.



04-Amazon e il Cloud

- Amazon, nata nel 1994 (online nel 1995), ha costruito data center globali per supportare milioni di clienti.
- Ha poi aperto la propria infrastruttura al pubblico come **Amazon Web Services (AWS)**:
 - 2006: primo servizio **S3 (Simple Storage Service)**.
 - Precedente alla nascita del termine “cloud”.



Principi di AWS

- Composto da **blocchi indipendenti e combinabili** (loose coupling).
- Accesso tramite **chiamate SOAP/REST**, incapsulabili in librerie per vari linguaggi.
- Disponibili strumenti CLI e grafici.
- Fornisce servizi sia **IaaS che PaaS**.



Costi

- **Pay-as-you-go**, con dimensioni principali:
 - Tempo (ore CPU, crediti)
 - Volume (GB trasferiti)
 - Conteggio (messaggi)
 - Spazio/tempo (GB/mese)
- Strumenti di:
 - metriche,
 - rendicontazione,
 - fatturazione.



Architettura AWS

- Strutturata in **regioni** (es. us-east-1) e **availability zones** (data center indipendenti).
- Region interconnesse via Internet, con scelta in base a **criteri legali e vicinanza utenti**.
- Availability zones con linee a **bassa latenza** tra loro, replicazione per affidabilità.



Sicurezza

- **Access identifier** + chiave privata.
- **ACL** per permessi oggetti.

- **IAM (Identity and Access Management)** per definire utenti e controlli di accesso granulari.



Servizi AWS principali

Computazione

- **EC2**, Elastic MapReduce, Elastic Container Service, LightSail.

Memorizzazione

- EBS, S3, Glacier, EFS.

Database

- Aurora, RDS (MySQL/Oracle), DynamoDB (NoSQL), ElastiCache.

Reti

- VPC, Route 53, Elastic Load Balancing.

Content Delivery

- CloudFront.

Servizi applicativi

- SWF, SQS, SES, SNS.
- Altri: MarketPlace, Snowball, servizi di migrazione, servizi serverless.



Dettaglio servizi

Amazon S3

- File system virtuale via web.
- Oggetti 1 byte - 5 TB, fault-tolerant, ridondante.
- Organizzati in **bucket** con nome univoco.
- Variante Glacier:
 - **Storage economico per dati acceduti raramente**, recupero lento, usato per backup.

Amazon CloudFront

- Content Delivery Network (stile Akamai).
- Replica oggetti su server distribuiti per velocizzare l'accesso.

Amazon EC2

- Fornisce **server virtuali scalabili** (istanze) da AMI.
- Tipologie istanze:
 - **On-demand, Reserved, Spot.**
- **Dischi non persistenti** → EBS se necessaria persistenza.
- Possibilità di **Elastic Load Balancing (ELB)**, auto scaling con CloudWatch.
- **Misurazione potenza:**
 - ECU (obsoleto) e **vCPU** (hyperthread di un core).
- Famiglie di istanze:
 - Uso generale: Tx, M4, M3
 - Calcolo: C4, C3
 - Memoria: R3
 - GPU: G2
 - Storage: I2, D2.

Database gestiti

- **Aurora:** DB relazionale ad alte prestazioni.
- **RDS:** MySQL, Oracle gestiti, con backup e replicazione.
- **DynamoDB:** NoSQL, storage strutturato.
- **SimpleDB:** NoSQL per piccole basi dati (fino a 10 GB/dominio).

Elastic MapReduce

- Implementazione di **MapReduce (Apache Hadoop)** su larga scala per applicazioni CPU-intensive.



Strumenti di gestione e sviluppo

Gestione

- **AWS Management Console** (web-based) per controllo interattivo di istanze, storage e DB.
- CLI in Java per script.
- Tool di terze parti (es. S3Fox).

Sviluppo

- API SOAP/REST.
- Librerie ufficiali per Android, iOS, Java, .NET, PHP, Ruby.



Esempi di utilizzo

- Hosting di **siti web statici**.
- Esempi reali (GIS Junta de Extremadura, runningforum.it con 90-95€/mese).
- Disponibile **Free Tier (12 mesi)** per sperimentare EC2, S3 e altri servizi.

• Conclusione

AWS fornisce **un'infrastruttura cloud scalabile e modulare** che consente:

- Computazione scalabile (EC2),
- Storage sicuro e ridondante (S3, Glacier),
- Distribuzione di contenuti rapida (CloudFront),
- Database gestiti SQL/NoSQL (RDS, DynamoDB),
- Strumenti di gestione e sviluppo semplificati,
- Pagamento a consumo con strumenti di monitoraggio.

05-Amaon EC2: cos'è

- **EC2 (Elastic Compute Cloud):** servizio AWS che fornisce infrastruttura di calcolo come server virtuali (istanze) con:
 - AMI (Amazon Machine Image) che contiene OS, server, DB, ambienti sviluppo.
 - Caratteristiche standardizzate (CPU, RAM, disco non persistente).
 - Scelta modalità pagamento (on-demand, reserved, spot) secondo le esigenze.

Scelta AMI

- Dipende dal software necessario in partenza (installabile anche manualmente).
- AMI base (Amazon Linux) è essenziale, senza Apache.
- Esistono **AMI standard, community e personalizzate**.
- Esempio: utilizzo **Ubuntu Linux su t2.micro** (free tier AWS).

Creazione e accesso alle istanze

- Creazione via **AWS Management Console**.
- Accesso via **SSH** con chiave privata generata da AWS.
- Comandi utili:
 - `ssh -i path_chiave ubuntu@ip`
 - `passwd, ls, ls -l, pwd, cd, mkdir`
 - `chmod a+x file, chmod a+r file, nano file` (editor, ctrl+x per uscire)
 - Tab (autocompletamento), freccia su (scorrimento comandi).

Uso di `sudo`

- Comandi amministrativi devono essere preceduti da `sudo` per motivi di sicurezza.
- Gli utenti abilitati a `sudo` si chiamano **sudoers**.
- Esempi:
 - Installazione pacchetti: `sudo apt-get install php`
 - Spegnimento/riavvio: `sudo shutdown -h now (-r per riavvio)`.

Tipologie di istanze

- **On-demand:** pay-as-you-go.
- **Reserved:** costo fisso + sconto sul costo orario.
- **Spot:** utilizzo risorse inutilizzate AWS a basso costo se l'app può essere interrotta.

Persistenza dei dati

- **Temporanee (non garantite):** istanze EC2.
- **Permanenti (garantite con backup/ridondanza):**
 - Volumi **EBS (Elastic Block Store)**.
 - Security Groups.
 - AMI su S3 o EBS.
 - Indirizzi IP elastici.
 - Load Balancer.

Costi EC2

Dipendono da:

- Tipo istanza.
- OS e software preinstallato.
- Esempio costi m3.large in Irlanda:
 - Linux: \$0.133/h
 - RHEL: \$0.206/h
 - SLES: \$0.246/h
 - Windows: \$0.258/h
 - Windows + SQL: \$0.703/h
- t2.nano:
 - On-demand: \$4.18/mese.
 - Reserved (1 anno anticipato): \$2.63/mese.
- Da aggiungere **costi EBS e banda**.

Costi EBS e banda

- EBS Standard: ~100 IOPS con picchi più alti.
- Provisioned IOPS su SSD: 30 IOPS/GB.
- Snapshot S3: backup persistente e compresso.
- Esempio:
 - EBS: €0.08/GB/mese + banda.
 - S3: €0.023/GB/mese + €0.0004/1000 GET.

CloudFormation

- Strumento per generare automaticamente infrastrutture complesse.
- Possibilità di usare template predefiniti o personalizzati.
- Esempio: avvio istanza EC2 con Wordpress e salvataggio dati su RDS in un solo click.

Istanze Spot

- Basate su **asta di risorse inutilizzate**.
- Funzionamento:
 - Si imposta un massimo di spesa oraria.
 - L'istanza parte quando il prezzo scende sotto il limite.
 - Si spegne se il prezzo sale sopra.
 - Necessaria un'immagine che parta e salvi dati automaticamente.
- Adatte per **computazioni batch non real-time**.

Funzioni aggiuntive

Elastic Load Balancer (ELB)

- Distribuisce traffico tra istanze in **una o più availability zones**.

CloudWatch

- Monitora risorse per riconfigurazioni in caso di utilizzo non ottimale.

Auto Scaling

- Con CloudWatch consente:
 - Scalabilità automatica (aumento/diminuzione istanze in base al carico).
 - Utilizzo risorse spot in modo ottimale per computazioni intensive.

Conclusione operativa

AWS EC2 consente di:

- Creare server virtuali scalabili su richiesta.
- Scegliere immagini AMI in base alle esigenze.
- Sfruttare **EBS per persistenza dati**, spot per risparmio, ELB per alta disponibilità.
- Automatizzare deployment complessi con **CloudFormation**.
- Gestire monitoraggio e scalabilità con **CloudWatch e Auto Scaling**.

06-EC2 Web Server

- **Creare e configurare un Web Server su EC2.**
- Attività pratiche:
 - Attivazione istanza EC2.
 - Accesso via SSH.
 - Installazione di Apache su Ubuntu.
 - Configurazione di file e privilegi per il web server.

Attivazione e accesso all'istanza EC2

- **Attivazione dell'istanza** tramite AWS Management Console.
- Accesso:
 - Via SSH online o con client SSH.
 - Uso della chiave privata generata da AWS.
- Comandi utili nella shell:
 - cd, cd .., cp, mv, ls, nano.
 - Freccia su per richiamare comandi precedenti, tab per autocompletamento.

Apache: concetti base

- Perché Apache:
 - Multiplatforma, modulare, personalizzabile.
- Funzionamento:
 1. Traduzione URI in percorso file.
 2. Verifica identità utente.
 3. Verifica privilegi di accesso.
 4. Altre forme di autenticazione.
 5. Determinazione tipo MIME della risorsa.
 6. "Fixups" per estensioni avanzate.

7. Invio della risposta al client.
 8. Registrazione eventi nei log.
- Ogni fase è gestita da moduli, sia nativi che di terze parti, con possibilità di crearne di personalizzati.

Struttura file e directory in Apache

- **Directory importanti:**
 - /var/www/ (radice dei documenti).
 - /var/www/html/
 - /var/www/cgi-bin/
 - /var/www/icons/
 - Se HTTPS:
 - /var/www/secure-html/
 - /var/www/secure-cgi/
- **Directory configurazione:**
 - /etc/apache2/
 - File principali: apache2.conf o httpd.conf, che includono altre configurazioni in:
 - conf-enabled, mods-enabled, sites-enabled.
- **Log:**
 - /var/log/apache2

Configurazione tipica Apache

Esempi di configurazione:

- ServerRoot /etc/apache2/
- ServerName pippo.dimi.uniud.it
- ServerAdmin pluto@localhost
- Listen *:80, Port 80
- DocumentRoot /var/www/html
- User apache, Group www-data
- Porte usate <1024 richiedono esecuzione da root.

Gestione processi:

- Configurabile in `httpd.conf`, `apache2.conf` o `/etc/apache2/mods-available/mpm_prefork.conf`:
 - `MinSpareServers`, `MaxSpareServers`, `StartServers`, `MaxClients`, `TimeOut`.
 - `KeepAlive On`, `MaxKeepAliveRequests`, `KeepAliveTimeout`.
 - `MaxRequestsPerChild` per HTTP/1.1.

🔑 Privilegi dei file

- File serviti da Apache devono essere:
 - Leggibili dall'utente con cui Apache è eseguito.
 - Directory devono essere attraversabili da tale utente.
- **Per pagine personali:**
 - Directory utente e `public_html` attraversabili da tutti.
 - File in `public_html` leggibili da tutti.
- **Permessi in Linux:**
 - `root`, `owner`, `group`, `world`.
 - Esempio permessi: `-rw-r--r--`
 - `rw-` (owner), `r--` (group), `r--` (world).
 - Permessi:
 - `r` lettura
 - `w` scrittura
 - `x` esecuzione (necessaria per "attraversare" directory).

Uso di `chmod`:

- Sintassi: `chmod [permessi] [file]`
 - Esempi:
 - `chmod 744 file` → `rwxr--r--`.
 - `chmod u=rwx,go=r file`.

- `chmod g+r file` aggiunge lettura al gruppo.

- Esempio operativo:
 - `cd /var/www`
 - `chmod -R a+rwX html`
 - `cd html`
 - `nano index.html`

📦 Installazione di Apache e PHP su Ubuntu

- Aggiornamento repository:
 - `sudo apt-get update`
- Installazione di Apache, PHP e moduli necessari:
 - `sudo apt-get install apache2 php libapache2-mod-php`

🧪 Alternative locali per test

Per testare un web server Apache in locale:

- **Windows/macOS/Linux:**
 - **XAMPP:** apacheFriends.org
- **macOS:**
 - Può includere Apache ma in alternativa:
 - **MAMP:** mamp.info
- **Linux:**
 - Usare il package manager:
 - Debian/Ubuntu: `sudo apt-get install apache2`.

• Sintesi operativa:

Con questo modulo:

- Sai **attivare un'istanza EC2**, accedervi via SSH.

- Installare e configurare **Apache su Ubuntu**.
- Gestire file e directory con **privilegi corretti per il web server**.
- Comprendere la **struttura di Apache, i moduli e le configurazioni**.
- Gestire processi, log, privilegi e installazioni in modo corretto.

🧩 07-Scripting lato server e PHP

- Le pagine web possono includere:
 - Codice HTML mescolato a **script interpretati** lato server.
 - Gli script sono inclusi tramite pseudo-tag (es. `<?php ... ?>`).
- Il **server HTTP** richiama un **modulo esterno** (es. PHP) per eseguire script riconosciuti tramite estensione o MIME type.
- L'esecuzione degli script avviene **ad ogni richiesta (stateless)** e l'output generato sostituisce gli pseudo-tag nel file.

⚙️ Esecuzione degli script PHP

- Gli script possono:
 - Essere parti di pagine HTML.
 - Costituire pagine interamente (come i CGI).
- Lo **stato della computazione persiste** tra più pseudo-tag nello stesso file.
- Output finale: HTML interpretato dal client.

PHP: Introduzione

- Nato come **Personal Home Page**, ora **PHP: Hypertext Preprocessor**.
- **Open source**, multiplatforma, versione 8.X.
- Disponibili librerie per:
 - Gestione stringhe, database, XML, cookie, immagini dinamiche.
- Utilizzato per siti come WordPress e CMS.

Sintassi e basi PHP

- **Linguaggio interpretato con sintassi derivata da C.**
- Può essere usato **procedurale o object-oriented (con limitazioni)**.
- Separatore istruzioni: ;
- Commenti:
 - // commento
 - /* commento multilinea */
- Tipi di dati:
 - **Scalari:** boolean, integer, float, string.
 - **Compositi:** array (indicizzati, associativi), oggetti.
 - Conversione automatica dei tipi se necessario.
- Variabili:
 - Prefisso \$, non richiedono dichiarazione preventiva.
 - Es. \$a = "test"; \$b = 18; \$c = \$b + 10;

Array in PHP

- **Array indicizzati:** \$a[0] = 1;
- **Array associativi:** \$a['nome'] = 'Mario';
- Tipo dei componenti eterogeneo.
- Accesso: \$a[0], \$a['nome'].

Variabili predefinite

- **\$_SERVER:** variabili server (es. QUERY_STRING, REMOTE_ADDR).
- **\$_GET, \$_POST:** parametri inviati via GET/POST.

Operatori utili

- Aritmetici: + - * /
- Logici: == != && || !
- Concatenazione stringhe: .
 - \$s = "Ciao " . "Mondo";

Strutture di controllo

- **Condizionali:**
- if (\$a == 10) { ... } else { ... }
- **Switch:**
- switch(\$var) {
- case 1: ... ; break;
- default: ... ;
- }
- **Cicli:**
- for (\$i=0; \$i<10; \$i++) { ... }
- while (\$cond) { ... }

Gestione form e parametri utente

- I form possono inviare dati via GET o POST.
- Accesso ai dati:
 - \$_GET['nome'], \$_POST['nome'].
- Gestione in un'unica pagina:
 - Se variabili settate → elaborazione.
 - Altrimenti → mostrare il form.

Funzioni PHP

- Definizione:

- function somma(\$a, \$b){ return \$a + \$b; }
- Variabili all'interno hanno **scope locale**.
- Variabili globali richiedono global \$var; per essere usate nella funzione.

Inclusione di file

- Per modularizzare:
 - include("file.php");
 - require("file.php");
 - Versioni _once evitano duplicazioni.
- Utilizzate per inserire:
 - Banner, menu, configurazioni comuni.

Funzioni utili di PHP

- echo: stampa a schermo (costrutto, senza parentesi).
- print(): simile a echo.
- phpinfo(): mostra configurazione PHP.
- isset(\$var): verifica esistenza variabile.
- die(\$msg): termina script e mostra messaggio.

Intestazioni HTTP

Controllo diretto delle intestazioni HTTP inviate al client tramite PHP.

- Di default, PHP invia:

Content-Type: text/html

- È possibile modificare o aggiungere intestazioni con la funzione header():

```
header("Location: http://www.uniud.it/");
exit; // consigliato dopo redirect
```

⚠ Regole fondamentali:

- Gli header **vanno inviati prima di qualsiasi output** (spazi, echo, HTML).
- In caso contrario, si riceve l'errore: *"Cannot modify header information – headers already sent..."*

📦 Content-Type: specificare il tipo di contenuto restituito

Serve a informare il browser su **come interpretare** il contenuto ricevuto.

Tipo di contenuto	Content-Type da usare	Descrizione
HTML	text/html	(predefinito) pagine web
Testo semplice	text/plain	testo non formattato
JSON	application/json	dati strutturati, usati in API
XML	application/xml	dati strutturati in formato XML
PDF	application/pdf	visualizzazione o download di PDF
JPEG	image/jpeg	immagini
PNG	image/png	immagini trasparenti
GIF	image/gif	immagini animate/statiche
CSV	text/csv	tabelle leggibili da Excel/LibreOffice
ZIP	application/zip	file compressi

Tipo di contenuto	Content-Type da usare	Descrizione
Download forzato	application/octet-stream + Content-Disposition	forza il salvataggio su disco

Esempi pratici:

```
// JSON
header("Content-Type: application/json");
echo json_encode(["ok" => true]);
```

```
// PDF
header("Content-Type: application/pdf");
// echo file_get_contents("file.pdf"); // oppure
output binario
```

```
// Download immagine JPEG
header("Content-Type: image/jpeg");
readfile("immagine.jpg");
```

```
// Forzare download file
header("Content-Type: application/octet-stream");
header("Content-Disposition: attachment;
filename=\"esempio.txt\"");
readfile("esempio.txt");
```

🍪 Cookie

- Si usano con `setcookie(nome, valore, scadenza)`, prima di qualsiasi output.
- Accessibili tramite `$_COOKIE['nome']`.
- Se valore nullo o scadenza passata → cancellazione cookie.
- Utilizzati per gestire sessioni utente e salvataggio dati persistenti.

08-PHP-Gallery

Realizzare una **galleria immagini dinamica in PHP**, con:

- **Pagina indice** con miniature e link.
- **Pagina di visualizzazione** della singola immagine, con navigazione precedente/successiva.
- Possibilità di **inserimento immagini tramite form**.
- **Creazione e gestione miniature** (GD).
- Accesso e gestione file tramite PHP.

Obiettivi di apprendimento

- Generazione dinamica di contenuto e link.
- Uso di funzioni PHP per:
 - Accesso al filesystem.
 - Gestione stringhe.
 - Ricezione dati via query string (`$_GET`).
 - Upload file tramite form (`$_FILES`).

Struttura dell'applicazione

- **Una sola galleria in /var/www/html/gallery.**
- **Cartelle:**
 - /gallery/immagini per immagini originali.
 - /gallery/thumb per le miniature.
- **File principali:**
 - index.php (pagina indice).
 - visualizza.php (pagina immagine singola).
 - inserisci.php, inserisci.html (upload immagini).
 - config.php (configurazione).
 - funzioni.php (funzioni utili).
 - pie_pagina.php (footer).
 - thumbnail.php (miniature on-the-fly, opzionale).

File config.php

Contiene configurazioni:

- define("DIR_IMMAGINI", "../immagini");
- define("DIR_THUMB", "../thumb");
- Dimensioni miniature: THUMB_X, THUMB_Y.
- Formati e MIME supportati:
 - .jpg, .png, .gif.

File funzioni.php

Contiene:

- caricaDirectory(\$dir): carica immagini da cartella.
- generaLinkImmagine(\$indice, \$file): genera link con miniatura.

- generaLinkTestuale(\$indice, \$testo): link testuale per navigazione.
- controllaFormato(\$nomefile): controlla estensione.
- controllaTipo(\$tipo): controlla MIME.

index.php (pagina indice)

- Include config.php e funzioni.php.
- Carica elenco immagini con caricaDirectory().
- Mostra le miniature con link a visualizza.php.
- Usa tabella per layout immagini con \$immagini_per_riga.

visualizza.php (visualizzazione immagine)

- Verifica presenza \$_GET["immagine"].
- Visualizza immagine a dimensione piena.
- Navigazione:
 - Link all'immagine precedente e successiva usando generaLinkTestuale().
- Include pie_pagina.php per i link di navigazione base.

inserisci.html (form upload)

- Form con method="post" enctype="multipart/form-data".
- Campi:
 - Username e password (opzionale).
 - File da caricare.
- Submit per inviare a inserisci.php.

inserisci.php (gestione upload)

- Verifica autenticazione se implementata.
- Gestisce \$_FILES:

- Controlla tipo MIME (controllaTipo()).
 - Controlla formato (controllaFormato()).
- Usa move_uploaded_file() per spostare l'immagine in DIR_IMMAGINI.
- **Generazione automatica della miniatura:**
 - Usa **GD** con:
 - imagecreatefromjpeg(), imagecreatefrompng(), imagecreatefromgif().
 - imagesx(), imagesy().
 - imagecreatetruecolor().
 - imagecopyresized() o imagecopyresampled() (più qualitativo).
 - imagepng() per salvare nella cartella thumb.
- Stampa messaggi di conferma o errore.

thumbnail.php (miniature on-the-fly, opzionale)

- Genera miniature al volo e le invia al browser senza salvarle.
- Usa **GD**:
 - getimagesize().
 - imagecreatefromjpeg() ecc.
 - imagecopyresized().
 - header("Content-type:image/png"); e imagepng() per output diretto.

Funzioni PHP e Filesystem utilizzate

- Gestione cartelle:
 - opendir(), readdir(), closedir().
 - is_dir(), is_file().
- Gestione file:
 - fopen(), fread(), fwrite(), fclose().
 - file_get_contents().
- Gestione stringhe:
 - Concatenazione con .
 - strpos(), strrpos().
- Superglobali:
 - \$_GET, \$_POST, \$_FILES.
- Upload:
 - move_uploaded_file().

Sicurezza

- L'applicazione base **permette upload a chiunque**, quindi:
 - Rimuovere/modificare inserisci.php al termine dei test.
 - Gestire permessi cartella immagini/ con chmod a+w immagini.
- Evitare di lasciare directory listing attivo sul server.
- Verificare **autenticazione anche nei file inclusi**.
- Usare sempre estensioni .php per i file inclusi per sicurezza.

GD Library

- Usata per:
 - Creazione di miniature.
 - Generazione dinamica di immagini.
 - Funzioni utilizzate:
 - getimagesize(),
 - imagecreatefromjpeg(),

imagecopyresampled(),
imagejpeg().

- Installazione:
- sudo apt-get install php-gd
- sudo systemctl restart apache2

09-PHP-eshop-sessioni

Creare un e-shop dinamico in PHP con:

- Visualizzazione **prodotti** con codice, nome, descrizione, prezzo, foto.
- Inserimento **prodotti** tramite form protetto da login.
- **Archivio prodotti** paginato.
- Introduzione **sessioni** per:
 - **Autenticazione utenti.**
 - **Gestione carrello.**

Versione 1 (senza carrello)

Semplificazioni

- Niente carrello né utenti registrati persistenti.
- Uso **filesystem** per memorizzare prodotti (shop.txt), in attesa di DBMS.

Struttura dell'applicazione

- Cartella negozio/:
 - index.php → home con ultimi prodotti.
 - inserisci.php → inserimento prodotto.
 - prodotti.php → archivio completo con paginazione.
 - config.php → configurazione.
 - funzioni.php → funzioni di gestione.
 - stile.php → CSS dinamico.

- db/shop.txt → memorizzazione prodotti.

index.php

- Visualizza gli ultimi \$NPRODOTTI con foto, descrizione, prezzo, data inserimento.
- Caricamento prodotti via leggi() da shop.txt.
- Include CSS dinamico con stile.php.

inserisci.php

- **Pagina bimodale:**
 - Se GET: mostra il form.
 - Se POST: esegue inserimento se utente e password validi (utenteValido()).
- Usa registra() per salvare il prodotto in shop.txt in ordine cronologico inverso.

prodotti.php

- Visualizza **archivio prodotti paginato**.
- Calcola \$pagine = ceil(\$numero / \$NPRODOTTI).
- Usa leggi() per caricare i prodotti richiesti.
- Navigazione tra pagine con ?pagina=n.

CSS dinamico (stile.php)

- Usa intestazione:

```
header("Content-type: text/css");
```

- Colori, font, stile menu e layout configurabili da config.php.

Memorizzazione prodotti (shop.txt)

- Ogni riga = un prodotto:

ID#Data/Ora#Nome#Descrizione#Prezzo#URI_foto

- Ordinati in ordine cronologico inverso per velocizzare le visualizzazioni frequenti.

Funzioni principali (funzioni.php)

- `registra($nome, $descr, $prezzo, $img)`
- `leggi($da, $n)`
- `numeroProdotti()`
- `utenteValido($utente, $password)`
- `rendiConforme($testo)` (normalizza input: `nl2br`, `htmlentities`, rimozione # e a-capo).

Sessioni e autenticazione

- Le sessioni consentono **persistenza dello stato** fra richieste.
- Si usano con:
- `session_start();`
- `$_SESSION["variabile"] = valore;`
- **Autenticazione:**
 - File `autentica.php` verifica credenziali da `utenti.txt`:

```
if ($login . ":" . $password == trim($riga))
```

- Una volta autenticato:

```
$_SESSION["autenticato"] = TRUE;  
$_SESSION["login"] = $login;
```

- Logout:

```
session_destroy();  
header("Location: form.php");
```

Versione “paranoica” elimina anche cookie `PHPSESSID` per pulizia completa.

Struttura autenticazione

- `index.php`: mostra contenuto protetto se loggato.
- `form.php`: form login.
- `autentica.php`: verifica login e imposta sessione.
- `logout.php`: logout e distruzione sessione.

Nota sicurezza: spostare `utenti.txt` fuori da directory web visibile per sicurezza.

Versione 2 (aggiunta del carrello con sessioni)

Funzionalità aggiunte:

- **Carrello tramite sessioni:**
 - Pulsante “compra” accanto a ciascun prodotto:

```
<form action="compra.php" method="get">  
  <input type="hidden" name="id" value="ID_PRODOTTO" />  
  <input type="submit" value="compra" />  
</form>
```

- `compra.php` aggiunge prodotto al carrello:

```
$_SESSION['carrello'][] = $id_prodotto;  
header("Location: carrello.php");
```

- `carrello.php` visualizza prodotti presenti in `$_SESSION['carrello']`.

Cosa aggiungere:

- Visualizzare nome e prezzo nel carrello.
- Calcolare il **totale** della spesa.

- Gestire quantità multiple dello stesso prodotto.
- Funzionalità per **svuotare o rimuovere singoli prodotti** dal carrello.

Note tecniche importanti

- **header():** usato per:

- dichiarare Content-type: text/css
- reindirizzamenti (Location:)
- gestione cache:
- `header('Cache-Control: no-cache');`
- `header('Pragma: no-cache');`
- `header('Expires: 0');`

- Funzioni PHP utilizzate:

- `file()`, `fopen()`, `fwrite()`, `fclose()`, `explode()`, `trim()`.
- `nl2br()`, `htmlentities()`, `stripslashes()`, `str_replace()`.
- `ceil()`, `floor()`, `round()`, `count()`.
- `$_GET`, `$_POST`, `$_SESSION`, `$_SERVER`.

- Sessioni:

- Consentono persistenza su server in modo sicuro.
- Essenziali per login e carrello e-shop.

- Sicurezza:

- File credenziali fuori dalla root web.
- Sessioni ben gestite con logout completo.
- Gestione corretta di input utente con `rendiConforme()` per prevenire XSS e garantire integrità dati.

- Con questo modulo ora sai:

- Creare un **e-shop in PHP** con visualizzazione, inserimento e archivio prodotti.
- Utilizzare **CSS dinamico** per configurazione aspetto.
- Gestire **autenticazione sicura** tramite sessioni.
- Implementare **carrello persistente** su sessione.
- Gestire e proteggere correttamente dati e file lato server.

10-PHP-OO-Composer

- I parametri di PHP sono definiti nel file **php.ini**:
 - Es: memoria massima per script, dimensione massima richieste HTTP, file di log.
- Si possono **sovrascrivere** alcune direttive nel singolo script tramite:
- `ini_set('display_errors', 1);`
- `ini_set('display_startup_errors', 1);`
- `error_reporting(E_ALL & ~E_NOTICE);` // mostra errori ma non notice
- Flag errori disponibili:
 - `E_ALL`, `E_ERROR`, `E_WARNING`, `E_PARSE`, `E_NOTICE`.
- In produzione si tende a mostrare meno errori.



PHP Object-Oriented Programming (OOP)

Perché usarla:

- Maggiore chiarezza, correttezza, manutenibilità del codice.
- Permette l'uso di **pattern avanzati**.
- Preferibile per progetti medi e grandi.

Sintassi:

```
class NomeClasse [extends ClasseMadre] {
    // proprietà
    // metodi (inclusi costruttori e distruttori)
}
```

- Visibilità:
 - `public` (default), `protected`, `private`.
- Non si può spezzare la definizione della classe tra tag `<?php ?>` (tranne dentro i metodi).

Esempio pratico: Classe libro

```
class libro {
    public $isbn, $titolo, $autore, $editore, $pagine;
    function __construct($i, $t, $a, $e, $p) {
        $this->isbn = $i;
        $this->titolo = $t;
        $this->autore = $a;
        $this->editore = $e;
        $this->pagine = $p;
    }
}

$l = new libro('8845290050', 'Il Signore degli Anelli', 'J.R.R. Tolkien', 'Bompiani', 1359);
echo $l->titolo;
Con metodi e distruttore:
function visualizza() {
    return $this->autore . ". <em>" . $this->titolo . "</em>." . $this->editore . ", pp. " . $this->pagine;
}
function __destruct() {
```

```
echo "<p>Sto distruggendo il libro con ISBN " . $this->isbn . "</p>";
}
```



Namespaces

Cosa sono:

- Simili ai namespaces XML.
- Risolvono le **collisioni tra nomi** di funzioni/classi/costanti (es. librerie diverse con lo stesso nome).
- Consigliati per **progetti pubblici o uso di librerie esterne**.

Sintassi:

All'inizio del file:

```
namespace TWC;
```

- Radice: `\`
- Supporto a sottospazi:
 - `namespace TWC\esempi\blog;`
- Utilizzo:
 - `TWC\funzione()`
 - `\funzione()` richiama la funzione globale
 - `\TWC\funzione()` equivale a `TWC\funzione()`



Gestione di librerie esterne in PHP

Metodi di installazione:

- Manuale (scaricare zip): scomodo per gestione dipendenze.
- PEAR: primo sistema di gestione pacchetti, ora in disuso.
- **Composer**: strumento attuale e raccomandato.

Composer

Cos'è:

- **Dependency Manager per PHP**, simile a npm (Node.js), bundler (Ruby).
- Funziona **per progetto**, installando librerie in vendor/.
- Opera via **riga di comando**, multiplatforma.

Principi di Composer:

- Favorisce ecosistema di **librerie facilmente installabili**.
- Le librerie:
 - Non creano file non richiesti.
 - Non emettono output/header.
 - Hanno stato locale.
 - Sono indipendenti dall'OS.
- Lo sviluppatore:
 - Definisce dipendenze.
 - Installa con un comando.
 - Include tutte le librerie con `require('vendor/autoload.php');`.

Repository principale:

- packagist.org



Usato pratico di Composer

Passaggi operativi:

• Creazione progetto:

- Creare directory (es. prova).
- Creare composer.json:

```
{
  "require": {
    "vendor/package": "versione"
  }
}
```

• Definizione versioni:

- Exact: "1.0.3"
- Range: ">=2.0 <4.0"
- Wildcard: "2.0.*"
- Next significant version:
 - "~1.3" = >=1.3 <2.0.0
 - "^1.3.1" = >=1.3.1 <2.0.0

• Installazione:

`composer install`

- Genera composer.lock:
 - Con composer.lock → librerie identiche a quelle installate dall'autore.
 - Senza → installa versioni più recenti compatibili.
- Per aggiornamenti:

`composer update`

• Uso nel codice:

```
require('vendor/autoload.php');
$obj = new vendor\package('parametri');
```

- Usa namespace per accedere a classi librerie:

```
$obj2 = new vendor2\package3();
```



Distribuzione del progetto con Composer

Per distribuire come package Composer:

- Aggiungere a composer.json:

```
{
```

```
"name": "TecWebCloud/Esempio",
"version": "1.0.2",
"require": {
  "vendor/package": "versione"
}
}
```

- Si possono indicare repository GitHub o personalizzati.



Sintesi finale:

- Configurare PHP e gestire errori per debug e produzione.
- Usare **programmazione ad oggetti in PHP** (classi, metodi, costruttori, distruttori).
- Organizzare codice con **Namespaces** per evitare conflitti e gestire meglio le librerie.
- Gestire le dipendenze in PHP con **Composer**:
 - Dichiarazione dipendenze in composer.json.
 - Installazione librerie via `composer install`.
 - Includere le librerie in blocco con `vendor/autoload.php`.
 - Preparare e distribuire progetti PHP strutturati, modulari e facilmente manutenibili.

11-SOA Cos'è un Servizio

- Entità **autonoma e indipendente**, in grado di:
 - Eseguire **funzioni semplici** (es. risposta a richieste).
 - Eseguire **processi complessi**.
- Indipendente dalla piattaforma, accessibile tramite **rete** con l'uso di **middleware**.
- Può essere **descritto, pubblicato, reperito e invocato**.

Service Orientation (SO)

Definizione:

- Paradigma di **progettazione e sviluppo** di applicazioni come **composizione di servizi indipendenti** e coordinati su rete.
- Consente di **fornire servizi accessibili e descritti** per la fruizione.

Principi della SO

- Standardized Service Contract:** contratto standard che descrive funzioni, canali, dati e policy (QoS).
- Service Loose Coupling:** servizi progettati indipendenti per ridurre le dipendenze.
- Service Abstraction:** nascondere dettagli interni per garantire accoppiamento debole.
- Service Reusability:** servizi progettati per il riuso, indipendenti dall'applicazione specifica.
- Service Autonomy:** controllo sul proprio ambiente per coerenza e affidabilità.
- Service Statelessness:** idealmente senza stato per prestazioni e scalabilità, ma possibili eccezioni

(es. DB).

- Service Discoverability:** servizi descritti con metadati per essere trovati e compresi.
- Service Composability:** servizi progettati per essere facilmente componibili.

Service Oriented Computing (SOC)

Disciplina che studia:

- Scienza, ricerca, tecnologia e progettazione dei servizi.
- Nasce dalla convergenza di sistemi distribuiti, middleware, ingegneria del software, protocolli, linguaggi e rappresentazione della conoscenza.

Obiettivi e benefici SOC

- Interoperabilità intrinseca:** capacità di scambio dati senza integrazione manuale.
- Federazione:** cooperazione tra risorse autonome.
- Diversificazione fornitori:** scelta tra fornitori diversi senza lock-in.
- Allineamento business-tecnologia:** astrazione dei livelli aziendali e rapida riconfigurazione dei servizi.
- Maggiore ROI, agilità organizzativa, riduzione del peso IT.**

Service Oriented Architecture (SOA)

Cos'è:

- Stile architetturale per supportare la **service orientation**, con componenti per:

- Pubblicazione, descrizione, reperimento e utilizzo dei servizi in rete.

- Indipendente dalla tecnologia** (ma spesso implementata tramite Web Services).

Implementazione SOA:

- Combinazione di tecnologie, API, infrastruttura.
- Possibile ad-hoc (Java, C#) o tramite tecnologie dedicate (Web Services).

Ruoli nella SOA

- Service Provider:** fornisce servizi e ne pubblica la descrizione in un **Service Registry**.
- Service Client:** utilizza servizi cercandoli nel **Registry**.
- Service Registry:** mantiene le descrizioni ricercabili dei servizi disponibili.

Operazioni nella SOA

Publish:

- Pubblicazione del servizio sul registry con dettagli su provider, modalità d'uso, caratteristiche e dettagli tecnici.

Find:

- Ricerca dei servizi nel registry tramite query su tipo, categoria, prezzo.
- Selezione manuale o automatica del servizio.

Bind:

- Invocazione del servizio usando i dettagli tecnici ottenuti.
- Connessione diretta al provider o tramite intermediari.



Esempio pratico SOA

- Azienda di parti in plastica che riceve ordini:
 - Servizio aggregatore riceve ordine.
 - Richiede disponibilità a magazzino.
 - Richiede spedizione al corriere.
 - Calcola prezzo e genera fattura.
- Ogni funzione può essere un **servizio indipendente** componibile.



Livelli di adozione SOA

- 1 SOA nell'organizzazione:** orchestrazione di applicazioni esistenti.
- 2 Organizzazione basata su SOA:** servizi comuni e standardizzati interni all'organizzazione.
- 3 SOA inter-organizzazioni:** processi collaborativi tra organizzazioni diverse.



Quality of Service (QoS)

Definizione:

Capacità del servizio di rispondere alle invocazioni secondo le aspettative del client.

Dimensioni:

- **Disponibilità:** uptime e TTR.

- **Accessibilità:** probabilità di invocazione riuscita.
- **Conformità agli standard e integrità.**
- **Performance:** throughput e latency.
- **Affidabilità:** funzionamento anche con problemi di rete/server.
- **Scalabilità:** gestione di variazioni di carico.
- **Sicurezza:** autenticazione, autorizzazione, integrità, confidenzialità.
- **Transazionalità:** coerenza nelle operazioni, definita in SLA.



SLA (Service Level Agreement)

- Contratto che definisce formalmente il livello di servizio tra provider e client.
- Contiene descrittori di qualità e penalità in caso di mancato rispetto.
- Può essere **statico o dinamico** per adattarsi ai cambiamenti.

Elementi SLA:

- Scopo, attori, periodo di validità.
- Ambito di competenza, vincoli, penalità.
- Servizi opzionali, esclusioni.
- Amministrazione con obiettivi misurabili.



SOA nella pratica

- Si vedranno:
 - Web Services standard (SOAP, WSDL).
 - Web Services REST.
- Implementazioni di requestor e provider verranno sperimentate.



Sintesi operativa:

Con questo modulo ora sai:

- Cos'è un **servizio** e il paradigma **SO (Service Orientation)**.
- I principi SO: contratti standard, loose coupling, astrazione, riuso, autonomia, statelessness, discoverability, composabilità.
- **SOC (Service Oriented Computing):** obiettivi e benefici (interoperabilità, federazione, diversificazione fornitori, allineamento business-IT).
- **SOA (Service Oriented Architecture):** cos'è, ruoli (provider, client, registry), operazioni (publish, find, bind).
- QoS e SLA per garantire affidabilità e qualità dei servizi.
- Esempi concreti e livelli di adozione SOA in azienda.

12-WS1 Cos'è un Web Service

Definizione (W3C):

- **Applicazione software identificata da URI**, con interfacce descritte tramite **artefatti XML**, che supporta interazione diretta via messaggi XML su **protocolli Internet** (HTTP, SMTP, FTP).

In sintesi:

- Componente software che esegue operazioni.
- Accessibile remotamente tramite **linguaggi XML**.
- Trasporto tramite protocolli di rete (principalmente HTTP).

Esempi:

- Motori di ricerca, quotazioni borsa, dizionari online.

Attori principali nei WS

- **Service Provider:** fornisce il servizio.
- **Service Client (Requestor):** utilizza il servizio.
- **Service Registry:** registro dei servizi disponibili.

Applicazioni tipiche

- **B2B:** integrazione tra sistemi informativi aziendali.
- **Interfacciamento sistemi legacy.**
- Integrazione funzionalità web in altre app (es.

ricerche da motori).
• Recupero e visualizzazione info complesse (spedizioni, voli).
• Alternativa strutturata allo “screen scraping”.

Tecnologie abilitanti dei WS

Standard di base:

- HTTP/SMTP/FTP: strato di trasporto.
- **XML/XMLSchema:** linguaggio base per descrizione e messaggistica.
- **SOAP:** protocollo di scambio dati.
- **WSDL:** descrizione dei servizi.
- **UDDI:** servizio di directory per WS.

Altri standard:

- **Composizione/collaborazione:** BPEL, WS-CDL.
- **Coordinamento/transazione:** WS-Coordination, WS-Transaction.
- **Sicurezza, policy, management:** WS-Security, WS-Policy, WS-Management.

Comunicazione nei WS

- Scambio messaggi **XML via HTTP**.
- Utilizzo **SOAP** per messaggi codificati:
 - Struttura:
 - **Envelope (busta):** radice del messaggio.
 - Header (opzionale): metadati.
 - Body: procedura da chiamare e parametri.
 - Attachments (opzionale): file binari via MIME.

SOAP (Simple Object Access Protocol)

Caratteristiche:

- Protocollo per scambio info in ambienti distribuiti.
- Nato da Microsoft e IBM, ora W3C.
- Supporta diversi scenari:
 - Fire-and-forget.
 - Request-response.
 - RPC (Remote Procedure Call).
 - Comunicazione asincrona.
 - Comunicazione con intermediari.
 - Invio dati binari con “SOAP with Attachments”.

Esempio SOAP:

Richiesta:

```
<Envelope
xmlns="http://schemas.xmlsoap.org/soap/envelope/"
>
  <Body>
    <getRate xmlns="urn:cambiovaluta">
      <country1>Italy</country1>
      <country2>USA</country2>
    </getRate>
  </Body>
</Envelope>
```

Equivale a:

getRate("Italy", "USA")

Risposta:

```
<Envelope
xmlns="http://schemas.xmlsoap.org/soap/envelope/"
>
  <Body>
    <getRateResponse xmlns="urn:cambiovaluta">
      <return>1.41</return>
    </getRateResponse>
  </Body>
</Envelope>
```

SOAP: vantaggi

- Semplicità e portabilità.
- Firewall-friendly.
- Standard aperto, interoperabile e riconosciuto.
- Indipendenza dalle infrastrutture.

SOAP: svantaggi

- ✗ Basato su HTTP, lento.
- ✗ Stateless (autenticazione necessaria ad ogni richiesta).
- ✗ Serializzazione per valore (duplicazione dati).
- ✗ XML ridondante e pesante.

 WSDL (Web Services Description Language)

Cos'è:

- Linguaggio XML per descrivere in modo **formale le API dei WS**, fornito dal provider e utilizzato dai client.

Struttura:

- **Types:** definizione tipi (XML-Schema).
- **Messages:** messaggi scambiati e tipi parametri.
- **Port Types:** definizione endpoint e operazioni.
- **Bindings:** protocolli/codifiche da usare.
- **Services:** definizione operazioni e indirizzi fisici.

Operazioni:

- Request-response (input/output).
- Solicit-response (output/input).
- Fire-and-forget (input).
- Notification (output).

Note:

- WSDL è ridondante ma consente di definire servizi complessi.
- I tool moderni (Axis, WSDL2Java, generatori PHP) generano automaticamente stub e skeleton dai WSDL.

WS-Policy

- Utilizzato per aggiungere **informazioni non funzionali** (es. QoS) alle descrizioni WSDL tramite un vocabolario esterno.

Web Services in PHP

- Da PHP 5 supporto **SOAP via libreria integrata** (richiede libxml >2.5).
- Da PHP 8 occorre installare manualmente:
sudo apt install -y php8.1-soap
- Uso tipico:
 - Parametri passati in **array associativi**.
 - Funzioni PHP per requestor e provider.

Esempio pratico in PHP:

Richiamo WS di cambio valuta:

```
$requestor = new  
SoapClient("https://ws.strikeiron.com/HouseofDev/  
currencyrates151?WSDL");  
echo "Cambio Euro - Dollaro: ";  
$cambio = $requestor->GetRate(array("EUR",  
"USD"));  
print($cambio->GetRateResponse);
```

Sintesi:

- Un **Web Service standard** usa XML, SOAP, WSDL e UDDI per offrire **servizi accessibili via Internet**.
 - SOAP gestisce la comunicazione strutturata (envelope, header, body, attachments).
 - WSDL descrive formalmente le operazioni, i tipi e le modalità di accesso al servizio.
 - WS-Policy estende WSDL con aspetti non funzionali.
 - PHP consente di creare e consumare WS con SoapClient e funzioni dedicate.
 - I WS semplificano l'integrazione tra applicazioni distribuite e forniscono interoperabilità, ma richiedono attenzione a performance e gestione dello stato.
-

13-REST-JSON, REST: cos'è

- **REST (Representational State Transfer):** paradigma generico (non solo per Web Services).
- **Principio base:**
 - Stato e funzionalità rappresentati da **risorse accessibili tramite URI univoci**.
 - Le risorse sono manipolate tramite i **metodi HTTP**:
 - POST → crea risorsa
 - GET → recupera risorsa
 - PUT → modifica risorsa
 - DELETE → cancella risorsa
- Richiede indicazione del tipo MIME (es. application/json, text/xml).

RESTful Web Services

- Ogni funzionalità esposta come **risorsa con URI significativa**:
 - Es: `http://www.motore.com/cerca/web/`, `http://www.motore.com/cerca/immagini/`
- Restituisce risorse in **XML o JSON**, da interpretare lato client.

JSON (JavaScript Object Notation)

- Formato di scambio dati leggero, indipendente dal linguaggio.
- Strutture:
 - **object:** coppie nome/valore (dizionario, struct, array associativo).
 - **array:** elenco ordinato di valori.

- I valori possono essere stringhe, numeri, booleani, null, oggetti o array.
- In PHP:
 - Codifica: `$json = json_encode($variabile);`
 - Decodifica: `$variabile = json_decode($json);`

Procedura REST

- Identificare le risorse da descrivere (cliente, ordine, spedizione).
- Mappare su URI significative (`www.sito.it/cliente/`).
- Associare metodi HTTP alle operazioni (CRUD).
- Passare parametri su URI o corpo richiesta.
- Client interpreta la risposta.

Esempi in PHP

JSON:

```
$vettore = array('primo'=>1, 'secondo'=>array('a','b','c'), 'terzo'=>"ultimo");  
$json = json_encode($vettore);  
echo $json;  
$nuovo = json_decode($json);  
var_dump($nuovo);
```

REST client minimale:

```
$endpoint = 'http://currencies.apps.grandtrunk.net/getlatest/';  
$risultato = file_get_contents($endpoint."EUR/GBP");  
echo $risultato;
```

REST vs SOAP

REST:

- Leggero, usa pienamente semantica HTTP.
- Risposte in XML o JSON.
- Sicurezza basata su HTTP.

- Non richiede WSDL ma necessita documentazione scritta.

SOAP:

- Più pesante, verbose.
- Standardizzato con discovery.
- Gestisce attachments.
- Richiede contratto formale (WSDL).

Caso di studio REST: Sistema informativo aziendale

Gestione di:

- **Clienti, Magazzino, Ordini, Fatture.**
- CRUD tramite:
 - GET `www.azienda.it/cliente/123` → legge cliente 123.
 - PUT `www.azienda.it/cliente/123` → modifica cliente 123.
 - DELETE `www.azienda.it/cliente/123` → cancella cliente 123.
 - POST `www.azienda.it/cliente/` → crea cliente.

Problematiche REST in PHP

- **Metodi PUT e DELETE non gestiti direttamente dai browser.**
- **Parsing del body della richiesta con:**

```
parse_str(file_get_contents('php://input'), $_PUT);
```

- Risposte HTTP significative (201 Created, 401 Unauthorized, 200 OK).

URL rewriting per REST

- Necessario per usare URI significative.
- Usare **mod_rewrite** di **Apache** configurando regole in `.htaccess`:
- RewriteEngine on
- RewriteRule `^cliente/(\w+)/?$ /azienda/service.php?res=cliente&id=$1 [L]`
- Abilitazione:
- `sudo a2enmod rewrite`
- `sudo systemctl restart apache2`

service.php REST handler in PHP

Script riceve richieste REST e gestisce risorse in base al metodo:

```
$metodo = $_SERVER['REQUEST_METHOD'];
$resorsa = $_GET['res'];
$id = $_GET['id'];
```

```
$_PUT = array();
if($metodo == 'PUT')
    parse_str(file_get_contents('php://input'),
    $_PUT);
```

```
switch($resorsa) {
    case 'cliente':
        $ok = gestisciCliente($id, $metodo);
        break;
    // idem per magazzino, ordine, fattura
}
header($ok['response']);
print(json_encode($ok, JSON_PRETTY_PRINT));
```

Funzioni `gestisciCliente`, `gestisciMagazzino`, `gestisciOrdine`, `gestisciFattura` gestiscono CRUD rispondendo con messaggi JSON/XML e header HTTP appropriati.

Risposte in XML o JSON

XML:

- Generabile come testo o con SimpleXML / DOM.

JSON:

- `header("Content-type: application/json");`
- `print(json_encode($dati));`

Client REST

- Usare `curl` da shell per simulare PUT/DELETE:
- `curl -X PUT http://localhost/azienda/cliente/13 -d "nome=Mario"`
- Usare tool online (Websniffer.cc, Postman) per test.

Documentazione dei servizi REST

- Non essendoci WSDL, si utilizza **OpenAPI / Swagger**:
 - Documentazione, testing e generazione codice.
 - Fornisce interfacce grafiche per test manuali.
 - Esempio: swagger.io

Sintesi finale:

- **REST = risorse con URI significative, manipolate via metodi HTTP (GET, POST, PUT, DELETE).**
- Usa risposte in JSON o XML, leggero, facile da integrare, richiede documentazione scritta.
- In PHP:

- `json_encode`, `json_decode` per JSON.
- `php://input` per gestire PUT/DELETE.
 - Utilizzare `.htaccess + mod_rewrite` per URL REST pulite.
 - OpenAPI/Swagger per documentazione e testing.

15-Cloud AWS S3: cos'è

- **Simple Storage Service**: quasi un **file system virtuale via Web**.
- Offre **storage ridondante, fault-tolerant**.
- **Oggetti**:
 - Dimensione 1 byte - 5 TB.
 - Accessibili via HTTP con nome univoco dentro un **bucket**.
 - Possiedono **ACL (Access Control List)**.
- **Bucket**:
 - Organizzano gli oggetti.
 - Max 100 bucket per account.
 - Nome universale univoco (es. `http://sitepoint-aws-cloud-book.s3.amazonaws.com/maggie.jpg`).

S3 Glacier

- Storage a **basso costo** per dati a cui si accede raramente.
- **Tempi di recupero molto lenti**.
- Ideale per **backup e archiviazione a lungo termine**.

SDK AWS per PHP

- **Versione 3, object-oriented, PHP >= 5.5.**
- Classi specifiche per ogni servizio AWS (S3, EC2, CloudFront...).
- Si installa preferibilmente con **Composer**:
- `composer require aws/aws-sdk-php`
- In alternativa scaricabile via `.phar` o `.zip`.

Problemi comuni SDK:

- **Timezone in php.ini** (`date.timezone = Europe/Berlin`).
- **Path inclusioni**, modificabile in `php.ini` in caso di problemi.
- **cURL**: installazione certificati per HTTPS (`apt-get install ca-certificates`).

Gestione delle credenziali AWS

- Ricerca automatica delle credenziali in:
 - Variabili d'ambiente.
 - File `~/.aws/credentials`.
- Ruolo IAM.
- Oppure passabili direttamente:

```
$credentials = new
Aws\Credentials\Credentials('key', 'secret');
$s3 = new Aws\S3\S3Client([
    'version' => 'latest',
    'region' => 'eu-south-1',
    'credentials' => $credentials
]);
```

Prerequisiti pratici

- Creare un bucket da **AWS Management Console** con almeno un oggetto per i test.

Esercizi in PHP con AWS S3

Esercizio 1: Elenco dei bucket

File `buckets.php`:

- Connessione a S3 con credenziali.
- Uso di:

```
$result = $s3->listBuckets();
foreach ($result['Buckets'] as $bucket){
    echo $bucket['Name'] . " : " .
    $bucket['CreationDate'];
}
```

- Debugging con `print_r()` e `var_dump()`.

Esercizio 2: Elenco oggetti nei bucket

File `objects1.php`:

- Stampa i bucket e l'elenco dei loro oggetti:

```
$elenco = $s3->ListObjects(['Bucket' =>
$bucket['Name']]);
foreach ($elenco->get('Contents') as $object){
    echo $object['Key'];
}
```

- Variante con iterator:

```
$iterator = $s3->getIterator('ListObjects',
['Bucket' => $bucket['Name']]);
foreach ($iterator as $object){
    echo $object['Key'];
}
```

Esercizio 3: Upload file su S3

File `fileS3.php`:

- Creazione bucket se non esiste:

```
if (!$s3->doesBucketExist($bucket)) {
    $s3->createBucket([...]);
    $s3->waitUntil('BucketExists', ['Bucket' =>
$bucket]);
}
Upload file:
$result = $s3->putObject([
    'Bucket' => $bucket,
    'Key' => basename($file),
    'SourceFile' => $file,
    'Metadata' => ['Content-Type' =>
'image/jpeg']
]);
```

Esercizio 4: Integrazione con CloudFront

- **CloudFront = Content Delivery Network di AWS** per migliorare accessibilità agli oggetti S3.
- Procedura da console: Vai in console AWS → CloudFront. Crea distribuzione selezionando un bucket. Configura opzioni (HTTP, log, dominio personalizzato).
 - Attendi la propagazione (`d3d5s2prd8s5ze.cloudfront.net`).

Attenzioni pratiche:

- S3 è a pagamento per ogni GET, PUT, ecc.: attenzione a cicli infiniti.
- Il tempo del server deve essere corretto (max 15 min di differenza).
 - Attendere che il bucket sia effettivamente creato prima di procedere.

Upload sicuro con Exponential Backoff:

- Se upload fallisce, attendere in modo crescente (1, 2, 4, 8 sec...) e riprovare.
- L'SDK AWS dalla versione 2 gestisce automaticamente il retry.

Altre funzioni S3

- **Multipart upload** per file grandi.
- **uploadDirectory** per grandi quantità di file.
- **downloadBucket** per download di interi bucket.

Sintesi:

- **AWS S3** fornisce **storage sicuro, scalabile, ridondante** via **web**.
- Oggetti organizzati in **bucket univoci** con ACL.
- Usabile via **SDK PHP**, preferibilmente tramite Composer.
- Supporta **operazioni di upload, download, elenco bucket e oggetti**.
- **S3 Glacier** per archiviazione a lungo termine a basso costo.
- **CloudFront** per distribuzione rapida e globale dei contenuti S3.
- Attenzione a **tempi del server, costi delle operazioni e gestione errori (backoff)**.

16-AWS-DynamoDB,

Introduzione a dati e database

- **Dati:** insiemi di simboli (es. “Mario Rossi”, “06 658976”), **non informazione finché non interpretati**.
- **Informazione:** interpretazione dei dati rispetto a una domanda (“chi è il direttore della banca?”).
- **Sistema informativo:** gestisce le informazioni di un’organizzazione, non sempre completamente digitalizzato.
- **Database:** insieme di dati utili, **stabili e persistenti**, risorsa fondamentale da proteggere.
- **DBMS:** software per gestire database, risolvendo problemi di:
 - Consistenza e duplicazione dati.
 - Condivisione tra più utenti.
 - Backup e recovery.
 - Efficienza e autenticazione.
- **Esempi DBMS:** Oracle, MySQL, PostgreSQL, MongoDB, Neo4j.

Modelli di dati

- **Modello gerarchico** (albero).
- **Modello a grafo**.
- **Modello relazionale:**
 - Organizza i dati in **tabelle (relazioni)** con righe (record) e colonne (attributi).
- **Modello a oggetti**.
- **NoSQL:**
 - “Not Only SQL”.
 - Non richiede schema fisso (schemaless).

- Supporta **attributi multivalore** e record eterogenei.
- Esempi: document-store, chiave-valore.

AWS e database

- **Amazon RDS:** database relazionali su AWS (Aurora, MySQL, PostgreSQL, Oracle, SQLServer), con gestione automatica server/backup.
- **Amazon DynamoDB:** database NoSQL su AWS:
 - **Serverless, scalabilità automatica.**
 - **Alto throughput**, cache in RAM.
 - Gestisce **documenti e chiave-valore**.
 - Dimensioni virtualmente illimitate.
 - Costi basati su item trasferiti.

DynamoDB: esercitazioni pratiche in PHP

Setup iniziale:

- Usare **AWS SDK per PHP (via Composer)**.
- Creazione **client DynamoDB**:

```
$client = new Aws\DynamoDb\DynamoDbClient([
    'version' => 'latest',
    'region' => 'eu-south-1',
    'credentials' => $credentials
]);
```

- Elencare tabelle:

```
$lista = $client->listTables();
foreach ($lista['TableNames'] as $t)
    echo $t . "\n";
```


Inserimento dati:

Esempio con rubrica:

```
$homer = [
  'nome' => ['S' => 'Homer'],
  'cognome' => ['S' => 'Simpson'],
  'indirizzo' =>
    ['S' => '742 Evergreen Terrace'],
  'luogo' => ['S' => 'Springfield'],
  'telefono' => ['N' => '123456789']
];
$rs = $client->putItem([
  'TableName' => $table,
  'Item' => $homer
]);
```

Stessa procedura per ulteriori record (es. Marge).

Query dati:

Ricerca per chiave:

```
$iterator = $client->getIterator('Query', [
  'TableName' => $table,
  'KeyConditions' => [
    'nome' => [
      'AttributeValueList' => [
        ['S' => 'Homer']
      ],
      'ComparisonOperator' => 'EQ'
    ]
  ]
]);
foreach ($iterator as $item) {
  echo $item['nome']['S'] . "\n";
}
```

Note:

- query() ricerca solo tramite chiavi.
- Limite: 1MB di risultati per volta.
- scan() permette filtri su tutti gli attributi.

Scan con filtro:

Esempio filtro su attributi:

```
$filtro = [
  'cognome' => [
    'AttributeValueList' => [
      ['S' => 'Simpson']
    ],
    'ComparisonOperator' => 'CONTAINS'
  ]
];
$lista = $client->scan([
  'TableName' => $table,
  'ScanFilter' => $filtro
]);
foreach ($lista['Items'] as $i) print_r($i);
```

Creazione tabella via PHP:

```
$client->createTable([
  'TableName' => "nuovatabella",
  'AttributeDefinitions' => [
    [
      'AttributeName' => 'nome',
      'AttributeType' => 'S'
    ],
    [
      'AttributeName' => 'cognome',
      'AttributeType' => 'S'
    ]
  ],
  'KeySchema' => [
    [
      'AttributeName' => 'nome',
      'KeyType' => 'HASH'
    ],
    [
      'AttributeName' => 'cognome',
      'KeyType' => 'RANGE'
    ]
  ],
  'ProvisionedThroughput' => [
    'ReadCapacityUnits' => 5,
    'WriteCapacityUnits' => 5
  ]
]);
```

Note pratiche per debug

• Funzioni utili:

- print_r() → stampa leggibile ma non eseguibile.
- var_dump() → stampa dettagliata con proprietà oggetti.
- var_export() → stampa in formato eseguibile.
 - Controllo memoria:
- memory_get_usage()
- memory_get_peak_usage()

Monitoraggio costi e attività

- Account Activity: controlla costi, gratuito nel free tier.
- Usage Reports:
 - Log attività in XML/CSV, apribili in Excel.
 - Analisi tramite tabelle pivot per prevedere i costi.
 - Attenzione a formati (punto/virgola nei decimali).

Sintesi finale:

- **DynamoDB = NoSQL AWS serverless**, scalabile, document-store e chiave-valore.
- Utilizzo pratico con AWS SDK PHP:

- Creazione tabelle.
- Inserimento (putItem).
- Query (getIterator, query).
- Scan con filtri.
 - Debug con print_r, var_dump, var_export.
- Controllo costi tramite AWS Activity e report.
 - Utile per gestire **dataset di grandi dimensioni, schemaless, multivalore**, con alta disponibilità e throughput.

17-Cloud-altro, Azure (Microsoft)

- **Cloud di Microsoft**, inizialmente **PaaS**, ora anche **IaaS**.
- Supporta **Windows e Linux**.
- **Prodotti principali:**
 - **Computazione:** VM scalabili, container (Kubernetes), piattaforma app web.
 - **Storage:** BLOB, SMB, archiviazione a basso costo.
 - **Database:** SQL Server, MySQL, PostgreSQL, NoSQL.
- **Free Tier simile ad AWS:**
 - Alcuni servizi gratuiti per 12 mesi, altri sempre gratuiti con limiti.
 - Link: [Azure Free Services](#).

Google Cloud

- Offre vari servizi cloud (computazione, storage, database, ML, ecc).
- **Free Tier:**
 - \$300 di credito per esplorazione servizi.
 - Alcuni servizi sempre gratuiti entro limiti.
 - Link: [Google Cloud Free Tier](#).

Heroku

Cos'è:

- **Servizio PaaS basato su Amazon EC2**, di proprietà **Salesforce.com**.

- Permette **sviluppo, pubblicazione, esecuzione di applicazioni** in vari linguaggi e framework.

Tool forniti:

- **Heroku Toolset:**
 - CLI Heroku.
 - **Foreman** per simulazione Heroku in locale.
 - **Git** per controllo versione e deploy.

 Ciclo di sviluppo:

- 1 Creazione nuovo repository Git.
- 2 Comunicazione a Heroku per creazione app usando stack **Cedar**.
- 3 **Push su Heroku** (simile a commit SVN).

 Cedar:

- Stack runtime di Heroku.
- Supporta la maggior parte dei linguaggi/framework moderni.
- Introduce **process model esteso** per web app:
 - Demoni per ogni tipo di processo.
 - Fork per aumentare prestazioni.

 Git e Heroku:

- Git usato per **deploy e gestione accessi:**
 - Si invitano collaboratori via email.
 - Possibilità di rimozione accessi.
 - Ogni sviluppatore deve conoscere Git.

 Ambienti multipli:

- Permette ambienti **staging e produzione** separati per test con i clienti senza intaccare il live.

Slug:

- **Al push:**
 - Heroku scarica l'app, rimuove file inutili (.git, log).
 - Imposta variabili ambiente.
 - Crea uno **slug** (max 500MB): immagine compressa dell'app pronta per l'esecuzione.
- Slug più piccoli sono più facilmente trasportabili tra **dyno**.

Dyno:

- **Singolo processo (web o worker).**
- Distribuiti su locazioni fisiche diverse:
 - 2 dyno = 2 processi su 2 macchine diverse.
 - **Scalabilità automatica senza riavvio:** slug copiato nei nuovi dyno.
 - Ogni dyno ha **512MB di RAM**.

Integrazione con S3:

- Heroku non pensato per servire grandi contenuti statici (limite slug).
- È **ospitato su AWS EC2**, per cui conviene usare **S3 per storage statico**, ma non è obbligatorio.

Add-ons:

- Numerosi **add-on integrabili**:
 - DB (Postgres, MongoDB, MySQL), monitoraggio, mailing, search engine, sicurezza.
- Aggiungibili via CLI o front-end web Heroku.
- Molti add-on hanno una versione base gratuita.

Pro e Contro Heroku:

• Pro:

- Alte prestazioni e scalabilità.
- Non richiede sistemista.
- Multi-linguaggio e multi-framework.
- Molti add-on.
- Costi contenuti.

• Contro:

- App devono essere progettate “con testa”.
- No tuning personalizzato (es. DB).
- Tutti gli sviluppatori devono conoscere Git.

Sintesi finale:

- **Azure**: Cloud Microsoft, PaaS + IaaS, storage, VM, database, free tier simile AWS.
- **Google Cloud**: \$300 di credito + free tier, vasta gamma servizi cloud.
- **Heroku**:
 - PaaS su AWS EC2.
 - Deploy semplificato via Git.
 - Usa dyno per scalabilità senza interruzioni.

- Usa slug per gestire app in modo leggero.
- Consigliato S3 per file statici.
- Molti add-on, gestione ambienti multipli.
- Richiede conoscenza Git per team.

18-Cloud-Lambda, Serverless Computing

Paradigma cloud

- che sposta l'esecuzione da **IaaS a PaaS**:
 - Il **provider gestisce i server e l'allocazione dinamica delle risorse**.
 - Il **client fornisce solo il codice** da eseguire.
 - **Pagamento pay-as-you-go**:
 - **EC2**: si paga a ore macchina accesa.
 - **Serverless**: si paga **per invocazione e risorse effettivamente usate**.

Deployment semplificato

- , ideale per applicazioni **piccole e ben definite**.

Event-driven

- Funzionamento basato su **eventi (trigger)** che attivano il codice:
 - Esempi:
 - Caricamento immagini/dati in un sistema.

- Click su un sito web.
- Dati ricevuti da sensori IoT.

Function as a Service (FaaS)

- Implementazione **serverless** per computazione:
 - Esecuzione di **funzioni autonome, stateless, svincolate dallo storage**.
- Ambienti **runtime predefiniti, meno flessibili di EC2**.
- Esempi:
 - **Google App Engine**: Java, PHP, Node.js, Python, Go, ecc.
 - **AWS Lambda**: Java, Node.js, C#, Python.

AWS Lambda

- **Servizio serverless di base su AWS**.
- Supporta **Java, Node.js, C#, Python** e altri linguaggi.
- Le funzioni si chiamano “**funzioni lambda**”.
- Gli eventi trigger possono provenire da:
 - Servizi AWS (S3, DynamoDB).
 - Web service via **AWS API Gateway**.
 - Eventi CloudFront (Lambda@Edge).

Costi AWS Lambda

- **Piano gratuito**:
 - 1 milione di richieste/mese.
 - 400.000 GB/s di tempo elaborazione/mese (es: 128MB allocati ≈ 860 ore/mese).
 - **Gratuito per sempre, non limitato a 12 mesi**.

- Oltre i limiti:
 - \$0,20 per milione di richieste.
 - \$0,00001667 per GB/s.

Casi d'uso

- Applicazioni **piccole, attivabili on-demand in risposta a eventi.**
- **Esecuzione parallela e scalabilità automatica.**

Orchestrazione con Step Functions

- **AWS Lambda + API Gateway = generazione di servizi.**
- **AWS Step Functions** permette di **orchestrare più servizi** in flussi di lavoro:
 - Sequenze.
 - Condizioni.
 - Esecuzioni parallele.
- Evita la scrittura di codice per la gestione dei flussi.

Composizione di Web Services

- Un WS composto è un insieme di service provider interconnessi, utilizzabili come un unico WS.
- Requisito: **compatibilità delle operazioni tra i WS.**
- **Modello di flusso** definisce la composizione.

Modalità:

• Orchestrazione:

- Controllata da un'unica entità che coordina i servizi.

• Coreografia:

- Scambio di messaggi tra entità indipendenti.
- Regole, contratti e convenzioni condivise.
- Nessuno controlla l'intero processo.
- Approccio collaborativo.

Esercizio pratico Lambda + S3

Obiettivo:

- Funzione Lambda che crea **automaticamente una miniatura** ogni volta che viene caricata un'immagine in una cartella specifica di un bucket S3.

Passaggi:

1 Permessi Lambda per accedere a S3:

- Creazione policy Lambda-S3 con permessi su S3.
- Creazione ruolo Lambda-S3-ruolo con:
 - Policy Lambda-S3.
 - Policy AWSLambdaBasicExecutionRole per i log su CloudWatch.

2 Creazione funzione Lambda:

- Scrivere codice in Python.
- Configurare trigger su S3 (evento ObjectCreated con prefisso immagini/).

- Caricare la funzione.

3 Attenzione:

- Se si legge e scrive nello stesso bucket, **evitare trigger ricorsivi** (miniatura che riattiva la funzione).

Codice Lambda:

Librerie usate:

- json, boto3 (SDK AWS per Python), os, urllib, PIL (Pillow per immagini).

Struttura:

- lambda_handler(event, context) riceve i dati dell'evento (bucket e chiave file).
- Scarica il file da S3 in /tmp/.
- Verifica formato file (.jpg, .jpeg, .png).
- Genera miniatura 120x120 con PIL.
- Carica la miniatura su S3 nella cartella miniature/.

Uso dei Layers:

- Permettono di aggiungere librerie extra.
- Usabili tramite ARN.
- Per Pillow:
 - arn:aws:lambda:eu-south-1:770693421928:layer:Klayers-p312-Pillow:2
- Layer personalizzati:
 - File zip con le librerie da noi preparate.

Test:

- Creare nel bucket:
 - Cartella immagini/.
 - Cartella miniature/.
- Caricare un'immagine in immagini/.
- Verificare che compaia il thumbnail in miniature/.
- Se non funziona, controllare i **log su CloudWatch (tab Monitora)**.



Sintesi finale

- **AWS Lambda = FaaS serverless di AWS: esecuzione codice senza gestire server, pagamento a invocazione.**
- Attivabile tramite **trigger/eventi** (S3, API Gateway, DynamoDB, ecc.).
- Codice eseguito in ambienti **stateless**.
- Perfetto per **applicazioni piccole, scalabili, parallelizzabili**.
- **AWS Step Functions** permette orchestrazione di flussi di Lambda senza codice aggiuntivo.
- **Coreografia vs Orchestrazione:** due modalità per la composizione di WS.
- **Esercizio pratico Lambda + S3:** creazione automatica di miniature al caricamento immagini con trigger, policy, layers e debug tramite CloudWatch.



19-AV-Assistenti Vocali:

introduzione

- Le interfacce vocali sono sempre più diffuse, su:

- Dispositivi tradizionali: smartphone, computer.
- Dispositivi dedicati: Alexa, Google Home, ecc.
- Obiettivo:
 - Semplificare l'accesso a servizi online **tramite voce**.
- Esempi:
 - Google Assistant, Alexa, Siri, Cortana, Bixby.



Sviluppare per assistenti vocali

- **Non sono sistemi chiusi:**
 - Si possono integrare nelle proprie applicazioni (es. app smartphone).
 - Si possono **aggiungere funzionalità accessibili dai dispositivi** tramite:
 - **Alexa Skills.**
 - **Google Actions.**

Struttura delle app vocali:

- Un'app vocal non riceve click o testo scritto, ma **parole/frasi riconosciute** da un motore AI che fa da intermediario.
- **Concetti base:**
 - **Pubblicazione:** rendere disponibile l'app.
 - **Invocazione:** comando che avvia l'app.
 - **Intento:** ciò che l'utente desidera fare (spesso richiede domande aggiuntive per completare i parametri).



Alexa Skills vs Google Actions

Alexa Skills

Google Actions

Oltre 100.000 disponibili (2021) Circa 5.000 disponibili (2019)

L'utente deve attivare esplicitamente la skill utenti una volta (anche acquistandola) pubblicate

Entrambe supportano app "codeless" (modelli supportano app predefiniti) "codeless"

- **Consigliato sviluppare per entrambe** per sfruttare la parte cognitiva dei vari sistemi.



Creazione di una Alexa Skill

- Utilizzo di **Alexa Developer Console:** <https://developer.amazon.com/alexa/console/ask>
- **Procedura:**
 - 1 Creare la skill (esercizio guidato, non pubblicata, utilizzabile solo col proprio account).
 - 2 Eseguire **"Build model"**.
 - 3 Sistemare il codice.
 - 4 Eseguire **"Deploy"**.
 - 5 Testare la skill (anche testuale).
- **Debugging:**
 - Attivare il **device log** durante il test.
 - Debug complicato, ma necessario.



Struttura di una skill

Handler

- Classi corrispondenti agli **intent**.
- Ogni intent ha un handler associato.
- Ogni handler implementa almeno:
 - `can_handle()`: verifica se può gestire l'intent.
 - `handle()`: esegue la risposta.
- Le risposte possono provenire da risorse esterne:
 - Web service.
 - Dispositivi.



Esempio di handler

```
class
argomentiIntentHandler(AbstractRequestHandler):
    def can_handle(self, handler_input):
        return
            ask_utils.is_intent_name("argomenti")
            (handler_input)

    def handle(self, handler_input):
        return (
            handler_input.response_builder
            .speak("PHP, cloud ed argomenti
                correlati")
            .ask("Vuoi sapere altro sul corso?")
            .response
        )
```



Slots

- Parametri passati dall'utente (possono essere completati in più passi).
- Esempio:
 - Intent: "che corsi ci sono al {annodicorso} anno".
 - Definizione di uno slot annodicorso con valori primo, secondo, terzo.
 - Recupero slot:

```
◦ anno =
ask_utils.request_util.get_slot_value(handle
r_input, "annodicorso")
```



Esempio completo di skill per il corso

TWC

- Intent definiti:
 - argomenti, docente, programma (programma usa slot annodicorso).
- **Risposte testuali** costruite dinamicamente.
- Skill costruita **da zero, non con template**.
- Possibilità di **output visivo** con SimpleCard.



Handlers utili

- **LaunchRequestHandler**: avvia la skill e dà il benvenuto.
- **SessionEndedRequestHandler**: gestisce la chiusura (es. salvataggio dati).
- **HelpIntentHandler**: risponde a richieste di aiuto.
- **CancelOrStopIntentHandler**: gestisce l'uscita ("Arrivederci!").
- **FallbackIntentHandler**: gestisce input non compresi.
- **CatchAllExceptionHandler**: cattura eccezioni generiche.
- **IntentReflectorHandler**: utile per debug, ripete l'intent attivato.



Costruzione della skill

- Si usa `SkillBuilder()` per aggiungere gli handlers in ordine:

```
sb = SkillBuilder()
sb.add_request_handler(LaunchRequestHandler())
sb.add_request_handler(argomentiIntentHandler())
```

```
sb.add_request_handler(programmaIntentHandler())
sb.add_request_handler(docenteIntentHandler())
sb.add_request_handler(HelpIntentHandler())
sb.add_request_handler(CancelOrStopIntentHandler(
))
sb.add_request_handler(FallbackIntentHandler())
sb.add_request_handler(SessionEndedRequestHandler(
))
sb.add_request_handler(IntentReflectorHandler())
# debug, ultimo
sb.add_exception_handler(CatchAllExceptionHandler(
))
lambda_handler = sb.lambda_handler()
```

- **Ordine importante**: gestionali in fondo.



Visualizzazione su schermo

- Con SimpleCard:

```
handler_input.response_builder
.speak(contenuto)
.set_card(SimpleCard(titolo, contenuto))
.ask("Vuoi sapere altro?")
.response
```



Sintesi finale:

- **Assistenti vocali**: interfacce vocali per semplificare l'accesso ai servizi.
- **Alexa Skills vs Google Actions**: strumenti per integrare app vocali, con modalità di pubblicazione differenti.

• Struttura skill:

- Intent, slots, handlers specifici.
- Possibilità di risposte dinamiche, output vocale e visivo.
 - **Sviluppo**:
 - Build, deploy, test.
 - Debug con device log e IntentReflectorHandler.
 - **Skill per TWC**: esempio pratico completo con intents, slots, risposte e output.



20-Cloud-IoT-BigData – Internet of Things

- **Definizione:** rete estesa a oggetti e luoghi fisici che **acquisiscono la capacità di comunicare e ricevere informazioni**, modificando il proprio funzionamento.
- **Applicazioni:** domotica, agricoltura, ambiente, robotica, biomedicale, smart city.
- **Smart City:**
 - Miglioramento qualità della vita attraverso connettività e partecipazione dei cittadini.
 - Sviluppo urbano sostenibile e inclusivo.

Tecnologie essenziali per IoT:

- **Miniaturizzazione:** Arduino, Raspberry, wearable.
- **Sensoristica:** ambientale, personale.
- **Comunicazioni a basso consumo/corto o lungo raggio:**
 - **Indirizzi:** IPv6 (2128 indirizzi) o spazi ridotti gestiti tramite gateway.



Architettura Cloud + IoT

- **AWS Lambda + API Gateway:**
 - Esecuzione reattiva a eventi da dispositivi IoT.
- **AWS IoT Core:**

- Registrazione dispositivi, storage dati, analytics, comunicazione sicura, aggregazione dispositivi.



Caso di studio: The Things Network (TTN)

- **LoRaWAN (Europa: 868MHz):** basso consumo, lunga distanza, bassa velocità.
- **TTN:** rete pubblica gratuita con limiti (max ~30 sec trasmissione/giorno).
- **Versione privata: The Things Industries (su AWS).**
- Integrazione con applicazioni esterne, incluse piattaforme IoT pronte.

Altre piattaforme e reti:

- Reti private (Vodafone, A2A, utility gas/acqua), reti pubbliche (TTN, LoraIoT), reti autogestite (loraserver).
- Flusso dati:
Nodo -> LoRa -> Gateway -> UDP/MQTT -> NetworkServer -> HTTPS/MQTT -> ApplicationServer



Dashboard e Analytics IoT

- Piattaforme indipendenti per visualizzazione e analisi dati sensori:
 - Grafana, Cayenne, Balena.
 - **Node-Red:**
 - Programmazione visuale IoT via web (basata su Node.js).
 - Integrabile con il cloud.
 - Comunità attiva per condivisione soluzioni.

- Offrono piani gratuiti con limiti su dispositivi e chiamate mensili.
- Compatibili con dispositivi noti e configurabili per altri.



Esempio locale: PollicIoT

- **Wearable:**
 - Basso consumo, GPS e accelerometro (posizione e movimenti).
 - Rilevamento cadute.
- **Infrastruttura LoRaWAN in FVG (Codroipo, Reana, Fagagna).**
- **Localizzazione in interni (Bluetooth).**
- **Sistema web per la gestione dell'assistito:**
 - Tracciamento posizione, geofencing, notifiche cadute, allarmi via SMS/Telegram.
- **Progetto finanziato dalla Regione FVG (POR FESR 2014-2020) per supporto a malati Alzheimer.**



Big Data

- Collezioni di dati troppo estese per analisi tradizionali:
 - Calcolo parallelo.
 - Cloud computing.
 - Statistica inferenziale.
- Dati non strutturati.
- Spesso derivano da interazione di fonti diverse.
- Tecniche:
 - Linked Data, ontologie, intelligenza artificiale.

Computazione in cloud

• Elasticità del cloud:

- Più potenza di calcolo in meno tempo.
- Utile per applicazioni ad alto consumo di calcolo non costanti (es. deep learning).

GPU in cloud:

- Offerte da tutti i cloud provider:
 - AWS EC2 con AMI per deep learning.
 - AWS SageMaker (servizio gestito per deep learning).
- Provider specializzati (paperspace.com).

GPU gratis o quasi:

• Google Colaboratory:

- Esecuzione notebook Jupyter su GPU nVidia Tesla (K80, 12GB).
- 12 ore di utilizzo interattivo gratuito.
- Versioni Pro disponibili per uso esteso.
 - **Kaggle:** simile a Colab, offre GPU per ML e challenge.



Storage per Big Data

- Database document-based (NoSQL).

• Data Lakes:

- Repositories centralizzati per dati strutturati, semi-strutturati e non strutturati.
- Archiviazione in formato nativo, gestione sicura.
- Esempi:
 - AWS Lake Formation.

- Azure Data Lake Storage.



Elaborazione e analisi dati in cloud

• Piattaforme PaaS e SaaS per analytics:

- AWS Kinesis, AWS IoT Analytics.
- Azure Analytics Services, Azure Data Lake Services.
- IBM Watson, AWS Rekognition.

• Sistemi AI specifici per:

- Immagini.
- Video.
- Dati sensoriali.



GPU: cloud o hardware?

Dipende da:

- Frequenza di utilizzo.
- Riuscita disponibilità.
- Ammortamento in relazione alla durata utile.
- Convenienza PaaS in caso di utilizzi specifici e occasionali.

Esempio: Nvidia A6000 $\approx$ 4000 ore di utilizzo per ammortamento, ma rischia di diventare obsoleta rapidamente.



Sintesi finale

- **IoT = estensione della rete agli oggetti fisici, applicabile in domotica, smart city, sanità.**
- **Tecnologie: miniaturizzazione, sensoristica, comunicazioni low-power (LoRa, Zigbee, NB-IoT).**

- **AWS Lambda, IoT Core, TTN per architetture IoT cloud.**

- Dashboard e analytics con Grafana, Cayenne, Node-Red per monitoraggio dati sensori.

- **Big Data = dataset di dimensioni e varietà tali da richiedere tecniche e strumenti dedicati.**

- Computazione elastica in cloud ideale per calcoli ad alta intensità (deep learning).

- GPU disponibili via cloud (AWS, Colab, Kaggle) per calcolo parallelo economico.

- Storage con data lake per gestire dati eterogenei e voluminosi.

- PaaS e SaaS per elaborazione e analisi (AWS Kinesis, Azure Analytics, Watson, Rekognition).
-